Latest updates: https://dl.acm.org/doi/10.1145/3772083

RESEARCH-ARTICLE

Uncovering Underexplored Runtime Behaviors in ROS2-Based Autonomous Systems

CHENGHAO FAN, Harbin Institute of Technology, Harbin, Heilongjiang, China
LANSHUN NIE, Harbin Institute of Technology, Harbin, Heilongjiang, China
JIACHENG ZHANG, Harbin Institute of Technology, Harbin, Heilongjiang, China
KUN DAI, Harbin Institute of Technology, Harbin, Heilongjiang, China
SHENGHAN GAO, Harbin Institute of Technology, Harbin, Heilongjiang, China
JING LI, New Jersey Institute of Technology, Newark, NJ, United States

Open Access Support provided by:
New Jersey Institute of Technology
Harbin Institute of Technology

PDF Download
3772083.pdf
01 March 2026
Total Citations: 0
Total Downloads: 151

Published: 17 October 2025
Accepted: 04 October 2025
Revised: 13 August 2025
Received: 03 April 2025

Citation in BibTeX format

Uncovering Underexplored Runtime Behaviors in ROS2-Based Autonomous Systems

CHENGHAO FAN*, Harbin Institute of Technology, Harbin, China
LANSHUN NIE*, Harbin Institute of Technology, Harbin, China
JIACHENG ZHANG, Harbin Institute of Technology, Harbin, China
KUN DAI, Harbin Institute of Technology, Harbin, China
SHENGHAN GAO, Harbin Institute of Technology, Harbin, China
JING LI†, New Jersey Institute of Technology, Newark, United States

Autonomous systems built on ROS2 are increasingly deployed in safety- and performance-critical domains such as autonomous driving and mobile robotics. While existing research has proposed various timing analyses and scheduling strategies for ROS2, many rely on simplified assumptions that do not hold in real-world applications. In this paper, we present a detailed empirical study of ROS2-based autonomous applications, uncovering underexplored runtime behaviors that significantly impact both real-time and functional performance. These include the importance of partial cause-effect chains, dynamic execution paths and timing variability, non-FIFO data access patterns, and computation threads uncontrolled by ROS2 executors. We extend an existing tracing tool to support Transform Library and ROS2's Action entity, enabling reconstruction and analysis of realistic cause-effect chains. Our findings are validated through experiments in simulated autonomous robot scenarios and a case study using the Autoware autonomous driving framework. Together, our results highlight the need for rethinking ROS2 modeling, scheduling and analysis to better reflect the realities of autonomous systems.

CCS Concepts: • **Software and its engineering** → **Real-time systems software**; **Real-time schedulability**; • **Computer systems organization** → *Real-time operating systems*; Robotics.

Additional Key Words and Phrases: ROS2, Real-time systems, Autonomous systems

1 Introduction

As robotic and autonomous systems continue to evolve in complexity and capability, ensuring their real-time correctness and functional reliability is paramount yet significantly more challenging, especially in domains such as autonomous driving, service robotics, and industrial automation. ROS2 (Robot Operating System 2) has emerged as the dominant middleware platform for these systems due to its modularity, scalability, and improved support for real-time execution, compared to its predecessor ROS1. Built atop the Data Distribution Service (DDS) middleware, ROS2 provides asynchronous communication primitives (e.g., publishers, subscribers, services,

*Both authors contributed equally to this research.
†Corresponding author.

Authors' Contact Information: Chenghao Fan, Harbin Institute of Technology, Harbin, Heilongjiang, China; e-mail: 23B903068@stu.hit.edu.cn; Lanshun Nie, Harbin Institute of Technology, Harbin, Heilongjiang, China; e-mail: nls@hit.edu.cn; Jiacheng Zhang, Harbin Institute of Technology, Harbin, Heilongjiang, China; e-mail: 23S003087@stu.hit.edu.cn; Kun Dai, Harbin Institute of Technology, Harbin, Heilongjiang, China; e-mail: 22S003028@stu.hit.edu.cn; Shenghan Gao, Harbin Institute of Technology, Harbin, Heilongjiang, China; e-mail: 24S003031@stu.hit.edu.cn; Jing Li, New Jersey Institute of Technology, Newark, New Jersey, United States; e-mail: jingli@njit.edu.

This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2577-6207/2025/10-ART
https://doi.org/10.1145/3772083

Just Accepted

and actions) and execution frameworks (e.g., executors and callback-based processing) that abstract away many system-level complexities from application developers.

Despite these design improvements, ROS2-based systems still exhibit timing unpredictability and lack full-system real-time guarantees, especially in real-world autonomous driving deployments. Existing analyses and scheduling strategies often overlook the practical nuances of ROS2 behavior. This work is motivated by the need to identify the gap between existing theoretical timing models and the runtime behaviors observed in autonomous systems, through an empirical approach.

Prior works have made important progress toward understanding the timing behavior of ROS2. A series of efforts [8, 9, 21, 23] model ROS2 applications as cause-effect chains, typically structured as directed acyclic graphs (DAGs) or processing chains, and use compositional response-time analysis (CPA) to estimate end-to-end worst-case reponse times. Other works focus on implementing priority-based scheduling strategies for callbacks or chains [2, 10, 19], or developing automatic online latency manager [7] to minimize response times of cause-effect chains. Additionally, system-level measurement tools such as `ros2_tracing` [5] have enabled tracing of runtime events and visualizing the flow of messages and callbacks in ROS2 applications.

However, several key limitations remain in the assumptions and capabilities of these existing analyses and tools. Most analyses focus on modeling and analyzing end-to-end cause-effect chains that span from sensor input to actuator output. However, in practice, many partial chains are equally critical and have their own real-time requirements. For example, delayed or missed updates to a global planner or localization module can compromise system behavior due to triggering recovery mechanism. Moreover, analyses typically assume a fixed set of chains, ignoring the dynamic nature of cause-effect chains in real-world applications. For instance, in the commonly used Navigation2, the chain structure changes depending on whether the robot is exploring, cruising, or recovering from failure. Similarly, different control or planning behaviors may activate or deactivate callbacks dynamically via mechanisms like behavior trees. This runtime variability in structure and timing makes static timing analysis inaccurate or overly pessimistic.

Data dependencies, as an indirect communication in addition to message passing, heavily exist in real-world ROS2-based applications. While existing works all assume FIFO communication semantics, where callbacks always consume message or data in buffer in FIFO order, non-FIFO data access patterns exist in complex ROS2 applications due to their functional semantics. A typical example is the Transform Library (TF), which enables coordinate transformation between reference frames. The TF buffer stores a history of transforms, and downstream callbacks can request transformations using older timestamps to ensure consistency with other sensor data. This breaks the FIFO assumption and can affect the metrics such as Maximum Reaction Time and Maximum Data Age [13].

In complex real-world autonomous systems, not all computation is managed by the ROS2 executor. Instead, many functionalities, especially those implemented using the ROS2 Action eneity, rely on long-running computations that are executed in threads spawned by the application, rather than by the ROS2 executor. For example, global planning modules in Navigation2 delegate heavy computation of planning to threads created by the application instead of ROS2. Similarly, modules such as computer vision pipelines (e.g., object detection using OpenCV) may spawn multiple application-level threads to perform image processing in parallel. These unmanaged threads are not subject to ROS2's executor scheduling semantics, yet they share CPU resources with other managed callbacks and interact via ROS2 messages or shared data structures. As a result, neglecting these threads in timing analysis leads to incomplete or inaccurate scheduling models, underestimating potential runtime interference and timing variability in critical cause-effect chains.

These gaps demonstrate a mismatch between the theoretical assumptions used in existing models and the real behaviors of deployed ROS2 systems. Motivated by these challenges, this work aims to empirically uncover these runtime behaviors, extend tooling support to capture them, and point out future research directions for real-time analysis and scheduling strategies for more realistic ROS2-based autonomous systems. Specifically, this work makes the following contributions.

Just Accepted

- Empirical Observations on Runtime Behaviors in ROS2 Applications: Through in-depth measurement and analysis of real-world ROS2-based systems, we uncover several previously underexplored runtime behaviors that have a substantial impact on both real-time performance and functional correctness. These include the importance of partial cause-effect chains beyond end-to-end chains, the runtime variability in both the structure and execution time of cause-effect chains, the presence of non-FIFO data access patterns in ROS2 communication, and the fact that not all computation threads are managed by the ROS2 executor, challenging common assumptions in existing timing analyses of ROS2 systems.
- Extension of the Tracing Toolchain for ROS2: We extend an existing ROS2 tracing and analysis framework `ros2_tracing` to support Transform Library TF-based intra-node communication and Action-based asynchronous execution, enabling accurate reconstruction of more complex and realistic cause-effect chains. These enhancements allow for finer-grained visibility into data dependencies and timing behavior in modern ROS2 applications.
- Experimental Validation in Robotic and Autonomous Driving Systems: We design and conduct comprehensive experiments in simulated autonomous robotic scenarios and perform a case study using Autoware, a representative autonomous driving system. The results empirically validate our observations and demonstrate their significance for real-time scheduling and functional outcomes in practical deployments.

2 Related Work

This section reviews the most relevant prior works on the tracing tools, benchmarking, timing analysis and real-time enhancements of ROS2.

Tracing Tools for ROS2. The `ros2_tracing` framework [5] introduced flexible, low-overhead instrumentation for ROS2, enabling collection of runtime execution data with minimal impact. It integrates with ROS2's tooling and uses the `LTTng tracer` to record events, including messages, callbacks, services, executor states, and lifecycle states, and enables all tracepoints added with overheads at the microsecond level. The captured ROS2 events can be merged with operating system traces to perform detailed performance analyses and pinpoint bottlenecks, overcoming the limitations of rosbag logging for timing/debugging. Recent work [4] has also extends `ros2_tracing` to extract and visualize message flows across distributed ROS2 systems.

Building on `ros2_tracing`, domain-specific tools like `Autoware Perf` [17] enables selective tracing of specific ROS2 nodes through manual input of node and callback dependency information. `Autoware Perf` focuses on analyzing end-to-end latency by measuring node execution time and communication delays between publish/subscribe pairs. Since it does not trace individual message instances, it estimates end-to-end latency analytically using the convolution of discretized probability distributions. Abaza et al. [1] introduce a tracing framework for ROS2 applications that utilizes extended Berkeley Packet Filter (eBPF) technology to probe functions within the ROS2 middleware without requiring direct instrumentation or recompilation of standard ROS2 libraries. This method allows for detailed modeling of complex behaviors such as message synchronization in sensor fusion and service requests from multiple clients.

Benchmarks for ROS2. Empirical benchmarking has quantified ROS2's real-time capabilities relative to purpose-built frameworks. For instance, one study [3] compares ROS2 with the Open Robot Control Software (OROCOS) framework running on vanilla and PREEMPT_RT patched Linux kernels under normal conditions and stress scenarios induced by the standard Linux `stress` tool. Kronauer et al. [14] specifically investigated ROS2 scalability by measuring end-to-end latency across long chains of nodes (mimicking a sensor-planner-actuator pipeline). In autonomous driving software, Betz et al. [6] applied a trace-based evaluation to the `Autoware.Universe` using `ros2_tracing`. They identified the end-to-end latencies of critical perception-planning-action chains and were able to pinpoint stages where most delay accumulates.

Just Accepted

Real-Time Analysis for ROS2 . Recognizing that ROS2's default executors are oblivious to deadlines, researchers have developed chain-aware scheduling models and analyses to bridge the gap between real-time theory and ROS2 practice. Casini et al. [9] performed the first formal response-time analysis of ROS2 processing chains. They modeled a ROS2 application's callbacks and topic communications as a directed acyclic graph (DAG) of tasks. Using Compositional Performance Analysis, they derived end-to-end worst-case response time bounds for chains in the callback DAG. Building on that, Tang et al. [21] introduced a more accurate model of ROS2 processing chains. By characterizing the processing windows between callback polling points and incorporating callback priorities, they significantly tightened the response-time bounds. Casini et al. [8] later extended their initial work by addressing the observation that ROS2 callbacks often have high execution-time variability. By modeling each callback using an execution-time curve instead of WCET and taking advantage of ROS2's default executor being starvation-free, they developed an improved response-time analysis that avoids unrealistically pessimistic queuing scenarios. Tang et al. [22] further improved upon Casini's refined analysis and introduced a more precise analysis on how callback activations saturate the polling windows to reduce pessimism.

While earlier works focused on the response time of a single chain from trigger to completion, Teper et al. [24] introduced Maximum Reaction Time (MRT) and Maximum Data Age (MDA) to capture timing across cause-effect chains spanning sensors to actuators. Analyzing MRT and MDA provides insight into not just the length of one processing chain, but also how fresh the data is that the system is acting upon. Tang et al. [23] highlighted that robotics developers sometimes use a data refreshing approach — for example, keeping only the latest sensor reading (overwriting older ones) to ensure the system always uses fresh data. They extended ROS2 timing analysis to this scenario by formulating how chains behave when intermediate data is refreshed. Their analysis computes the maximum reaction time and data age under data-refreshing.

Analyzing Synchronization in ROS2. Li et al. [15] turned attention to ROS2's `message_filters` — specifically, the Approximate Time synchronizer which combines messages from different topics (e.g., sensors) that have close timestamps. They modeled ROS2's approximate time synchronization policy and derived the worst-case time disparity between messages in a synchronized set. Their follow-up work [16] extended the analysis to cover the latency impact of the synchronization process itself. They noted that using a synchronizer not only can output misaligned timestamps, but also adds delay. This work establishes upper bounds on how much additional delay a synchronizer can contribute to an end-to-end chain. Sun et al. [20] concentrated on improving ROS2's Approximate Time synchronizer and identified that the default ApproximateTime policy can sometimes cause large delays or even degraded synchronization over successive cycles. Thus, they proposed an improved synchronizer and proved that this synchronizer guarantees an upper bound on the time disparity and reduces the latency of synchronization.

Improvement to ROS2 Scheduling. One major effort is ROS-Llama [7], an automatic online latency manager for ROS2. ROS-Llama lets system integrators specify a maximum latency for each important cause-effect chain and then transparently instruments the running ROS2 system to model its threads and callbacks. At runtime, it adjusts thread priorities or budgets to meet the chain latency goals, requiring little real-time expertise from the user. It also implements graceful degradation: if not all deadlines can be met during overload, less critical chains are dynamically deprioritized to protect the timing of crucial ones. To address the fact that the default ROS2 executor schedules callbacks in FIFO order and ignores the different criticality of tasks, PiCAS [10], a priority-driven scheduler for ROS2, was developed by modifying the ROS2 executor to strictly follow assigned priorities. Instead of fixed priorities, Arafat et al. [2] explored a dynamic priority approach by applying Earliest Deadline First (EDF) scheduling to ROS2. They extended the ROS2 executor to maintain a deadline-sorted ready queue for individual callbacks. The executor always picks the callback with the earliest deadline to run next, effectively implementing EDF at the user level.

Just Accepted

3 Background

This section provides background information necessary to understand the context and technical foundations of this work. We first introduce key concepts and mechanisms in ROS2, including its basic entities, executor scheduling, and the ROS2 Action entity. We then describe how cause-effect chains are modeled in ROS2 systems and highlight critical components such as the TF transform library, the Navigation2 library, and the use of behavior trees in autonomous systems. These components and abstractions play an essential role in shaping both the functional structure and timing behavior of ROS2-based autonomous systems.

3.1 ROS2 Basic Entities and Executor Scheduling

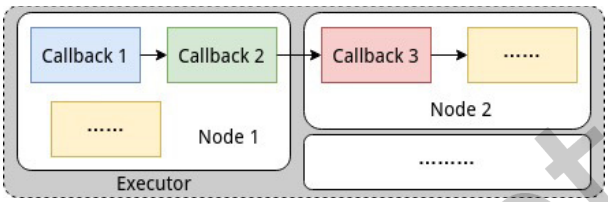


Fig. 1. Basic entities in ROS 2 and their relationships. Each node contains one or multiple callbacks. An executor manages one or more nodes and is responsible for scheduling and executing their callbacks.

In ROS 2, ***callbacks*** are the atomic schedulable units of execution. A callback is a specific piece of logic and computation to be run in response to an event, such as a new message or a timer. Callbacks can be triggered by various ROS2 entities: publisher–subscriber pairs activate subscription callbacks upon message arrival, client–service pairs trigger service callbacks when requests are received or responses are returned, and action clients and servers activate goal, cancel, feedback, or result callbacks in long-running interactions. As illustrated in Fig. 1, these callbacks are organized within a ***node***. Each node may define and register multiple callbacks associated with different communication interfaces. The execution priority between callback types is in the order of timers, subscriptions, service handlers, and clients. The execution ordering among callbacks of the same type is determined by the order in which the callbacks are registered during node initialization — the earlier a callback is added, the higher its execution priority. Timers are periodically activated based on the time of their creation and its specified period. While nodes contain callbacks, the actual execution is delegated to an ***executor***. ROS2 provides two main types of executors: the single-threaded executor, which processes one callback at a time, and the multi-threaded executor, which can schedule multiple callbacks in parallel using multiple worker threads. In this paper, we focus on the single-threaded executor and use multiple executors when having multiple cores due to its more deterministic scheduling characteristics.

The executor is responsible for managing the execution of all callbacks attached to it. Each callback transitions through four states: idle, activated, sampled, and running. By default, a callback remains idle until it is activated by an upstream event (e.g., a new message from a publisher). When the executor finds its ReadySet empty, it scans all attached entities to detect activated callbacks, moves them into the ReadySet, and marks them as sampled, as shown in Fig. 2. The executor then selects one sampled callback and executes it, transitioning it into the running state. The ROS2 executor uses a *non-preemptive* scheduling model — once a callback starts executing, it runs to completion without interruption. Within the ReadySet, callbacks of different types (e.g., timers, subscriptions, services) are selected based on an implicit priority defined by ROS 2's scheduling policy. Although not user-configurable, this priority order affects which callbacks are preferred when multiple are ready, influencing system timing behavior.

Just Accepted

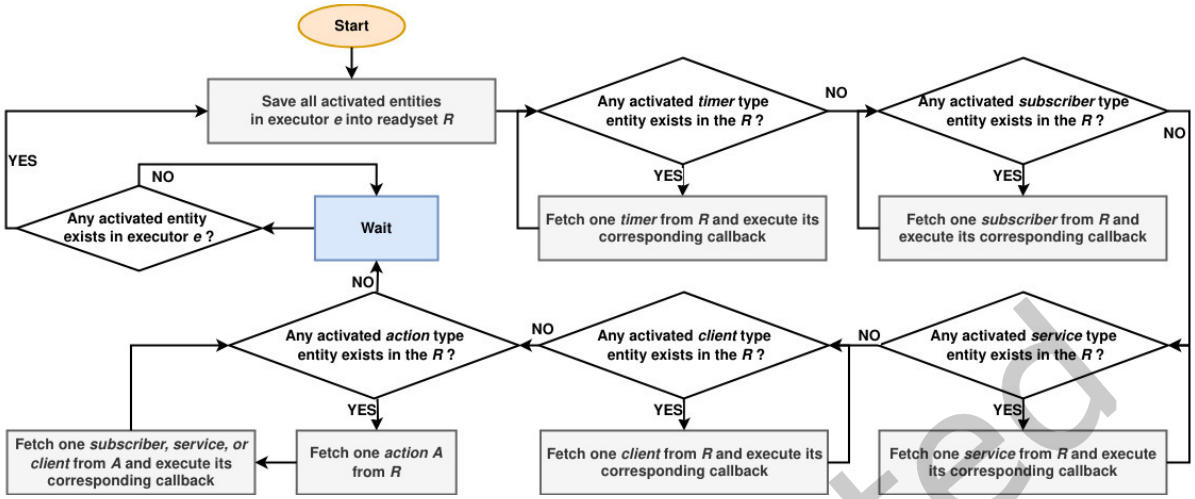


Fig. 2. Scheduling mechanism in a ROS 2 executor. Activated callbacks are first collected into the ReadySet. The executor then selects callbacks from the ReadySet in a fixed implicit priority order: timer, subscriber, service, client, and action. The selected callback is executed non-preemptively until completion.

3.2 ROS2 Action Entity

In autonomous system applications, the ROS2 ***Action*** interface is widely used to implement critical long-running functionalities and plays a key role in cause-effect chains. An action consists of two core components: the Action Client and the Action Server, supporting goal management, feedback, and result reporting. Internally, it encapsulates multiple ROS2 primitives (e.g., publishers, subscribers, clients, and services), enabling seamless interaction with other callbacks for coordinating the execution of long-running tasks.

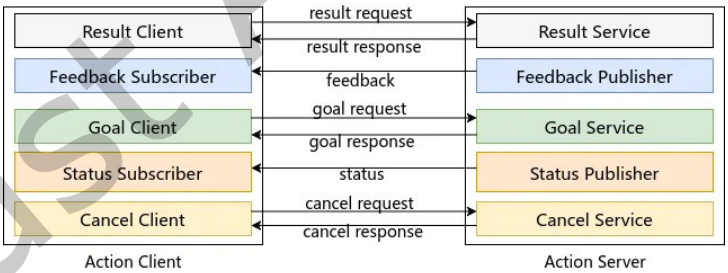


Fig. 3. Structure and behavior of the ROS2 Action entity. The Action Client comprises a Result Client, Feedback Subscriber, Goal Client, Status Subscriber, and Cancel Client, while the Action Server includes a Result Service, Feedback Publisher, Goal Service, Status Publisher, and Cancel Service. The interaction involves result/goal/cancellation request and response, and feedback/status pub-sub during execution.

Fig. 3 shows the behavior of the ROS2 Action entity. Specifically, at runtime, a callback can trigger an Action by using the Goal Client to send a goal request to the Action Server. The Action Server responds with an acknowledgment and begins executing the associated long-running task. During task execution, the Action Server can continuously publish feedback messages via its internal Feedback Publisher, which are received by

Just Accepted

the Action Client. In addition, the Status Publisher on the Action Server can update its execution status to the Status Subscriber on the Action Client. Upon task completion, the Action Client can use the Result Client to request the final outcome of the task from Action Server. The Action Client can also invoke the Cancel Client to send a cancellation request to the Cancel Service on the Action Server. In response, the Action Server stops the ongoing long-running task and returns a confirmation. Callback scheduling within the Action entity follows predefined priorities. On the Action Client side, the callback priority (from high to low) is: Feedback Subscriber, Status Subscriber, Goal Client, Result Client, and Cancel Client. On the Action Server side, the priority order is: Goal Service, Cancel Service, and Result Service.

Although the ROS2 Action entity is implemented atop existing callback mechanisms, its multi-phase, stateful interactions introduce a unique layer of behavioral complexity that is not covered by existing timing models or analyses. To date, no prior work has explicitly modeled the timing behavior of ROS2 Actions nor their impact within end-to-end chains. In this work, we empirically demonstrate how actions behave and affect timing in real-world autonomous system applications.

3.3 Cause-Effect Chains in ROS2 Applications

Prior works [8, 9, 21, 22] model the computation in ROS2-based applications as a directed acyclic graph (DAG), focusing primarily on the real-time performance of the set of cause-effect chains within the DAG. Each ***cause-effect chain*** consists of a sequence of callbacks with data and control dependencies between them. These chains represent how input data (e.g., from sensors) propagates through processing nodes and eventually influences outputs (e.g., actuator commands). Most cause-effect chains are periodically activated, with their activation frequency determined by a source callback. For example, if the chain originates from a Timer callback, its period equals the timer interval. Alternatively, some chains are triggered by external events, such as asynchronous user input or environmental changes; these are considered non-periodic cause-effect chains.

It is important to note that dependencies between callbacks within a cause-effect chain can be established via two different mechanisms: inter-node communication and intra-node communication.

- ***Inter-node communication*** occurs between entities utilizing DDS (Data Distribution Service) for communication in ROS2, including: publisher-subscriber, client-service, as well as Action Client and Action Server mechanisms. These dependencies commonly span across different nodes. DDS is a middleware designed for real-time communication in distributed systems. In ROS2, DDS can transmit information through both network and shared memory.
- ***Intra-node communication*** occurs through a message-passing process that does not require DDS, namely the indirect mechanism, such as an internal buffer within the same node or a shared buffer between nodes. In this case, the message is temporarily stored and will only be consumed when the downstream callback is activated–usually by a separate timer or an external event.

Tang et al. [22] classified cause-effect chains composed of inter-node communication as trigger chains, while those using intra-node communication are termed data chains.

A typical ***end-to-end*** cause-effect chain starts from a sensor node (e.g., LiDAR, camera) and ends at an actuator node (e.g., motors, wheels). In such chains, a timer callback on the sensor node periodically samples the environment, and a subscriber callback in the actuator node receives control commands. It is commonly assumed that a control command remains effective on the actuator until the next command arrives. The ***response time*** of a single execution instance of a cause-effect chain is defined as $t_e - t_s$, where t_s is the start time of the first callback and t_e is the completion time of the last callback in the chain.

To characterize system responsiveness and data freshness, two metrics have been introduced: Maximum Reaction Time and Maximum Data Age. Suppose an external event occur at time t_0, but the sensor fails to detect it immediately. Instead, it is captured at the next sampling point t_1, and its effect is applied to the actuator at time

Just Accepted

t_2. Then, the ***Maximum Reaction Time*** is defined as $t_2 - t_0$. Alternatively, suppose the event is captured by the sensor at t'_0, takes effect by the actuator at t'_1, and this effect is ultimately superseded at t'_2. The ***Maximum Data Age*** is defined as $t'_2 - t'_0$.

Crucially, these metrics signify distinct concepts. Response time denotes the latency of a message as it passes from the starting callback to the destination callback in a cause-effect chain. Maximum Reaction Time signifies the longest delay between the occurrence of an external event to the moment it exerts its influence on actuation. Maximum Data Age refers to the maximum duration from the detection of an external event by a sensor until the resultant effect on actuators ceases.

3.4 Transform Library (TF), Navigation2, and Behavior Trees in Autonomous Systems

In ROS2-based autonomous systems, the Transform Library (TF), Navigation2, and Behavior Trees are independent yet complementary components, each fulfilling distinct roles while working together to enable reliable autonomous behavior.

The ***Transform Library*** (***TF***) [12] is widely used package to manage and query spatial relationships between different coordinate frames, crucial for understanding the robot's position and the environment's layout. In ROS2, the updated version of this library is called TF2; for consistency, we refer to it simply as TF throughout the remainder of this paper. TF maintains ***TF buffers***, which continuously listen to transformation messages published by various sensors and subsystems (e.g., LiDAR, IMU, odometry). These messages describe how different coordinate frames (e.g., base_link, odom, lidar) are positioned and oriented relative to each other over time. With access to a TF buffer, a point p_t that is represented in coordinate frame A at a past timestamp t can be automatically converted into its equivalent coordinates p'_t in coordinate frame B. This functionality is critical in real-world systems, where multiple coordinate frames are defined and evolve over time.

Note that the TF library handles coordinate frame transformations through a distributed architecture: each node that requires access to transform data maintains its own local TF buffer, which is automatically synchronized globally. This distributed design ensures consistency across ROS2 nodes while minimizing access latency, as each callback can retrieve transform data directly from its own local buffer without querying a centralized structure. Navigation2 nodes that perform coordinate transformations–such as those involved in planning and control–leverage TF by having their own TF buffer instances. TF is also widely used in other ROS2-based autonomous systems, including Autoware where TF coordinates transformations between the map and lidar frames.

Navigation2 [18] is the primary navigation stack used in modern ROS2-based autonomous systems. It provides a comprehensive suite of modules for perception, planning, control, localization, visualization, and more. In a typical application, a user or autonomous task planner specifies a target position on a map, and Navigation2 automatically generates a global plan, performs real-time obstacle avoidance, and drives the robot toward the goal location. The stack makes heavy use of TF to compute transforms between map, odometry, and robot base frames, and relies on Action entities to coordinate long-running tasks such as path planning and goal execution.

To achieve intelligent decision-making and modular behavior control, the ***Behavior Tree*** (***BT***) is widely adopted in autonomous applications. A Behavior Tree is a hierarchical structure where leaf nodes correspond to concrete robot actions or behaviors, while composite nodes define control flow logic. Navigation2 integrates BTs as its default mechanism for task execution, enabling flexible composition and dynamic switching between behaviors. For example, one can define a high-level navigation behavior, then selects a planner, and finally sends a navigation goal, with fallback and recovery behaviors as needed. In practice, a robotic application will tick the behavior tree at a fixed frequency, during which the tree is traversed from the root, and its state is updated. This periodic execution allows the robot to react to changes in the environment, update its behavior, and make preemptive transitions between tasks based on real-time conditions.

Just Accepted

In summary, Navigation2, the navigation stack for ROS2 applications, coordinates motion planning and execution, while the Behavior Tree serves as the high-level logic and state controller. The behavior tree governs when and how Navigation2 transitions between tasks such as Global Planning, Local Planning, and Recovery. While TF provides the spatial context and Navigation2 executes the core navigation functionalities, the behavior tree ensures these components are invoked at the right time and in the correct sequence, enabling flexible and robust autonomous behavior across a variety of mission scenarios.

4 Empirical Observations from Real-World ROS2-based Autonomous Systems

In this section, we present a series of empirical observations obtained through our measurement and analysis of ROS2-based autonomous robotic applications and an autonomous driving system. These observations reveal runtime behaviors that are either not discussed or only briefly studied in prior work, yet they have a substantial impact on both the real-time and functional performance of the overall system. By highlighting these findings–grounded in real-world traces and execution data–we aim to guide future research toward addressing overlooked aspects of ROS2 behavior.

4.1 Importance of Partial Cause-Effect Chains in ROS2 Applications

As discussed in Section 2, prior work on timing analysis in ROS2 has predominantly focused on end-to-end cause-effect chains, which typically start from sensors (such as LiDARs or cameras) and end at actuators (such as wheels or motors). Ensuring that these chains meet their real-time constraints is undeniably essential for the safety and responsiveness of autonomous systems. Notably, as detailed in Section 3.3, such end-to-end chains are generally triggered by timer callbacks, which initiate periodic sensing and processing cycles.

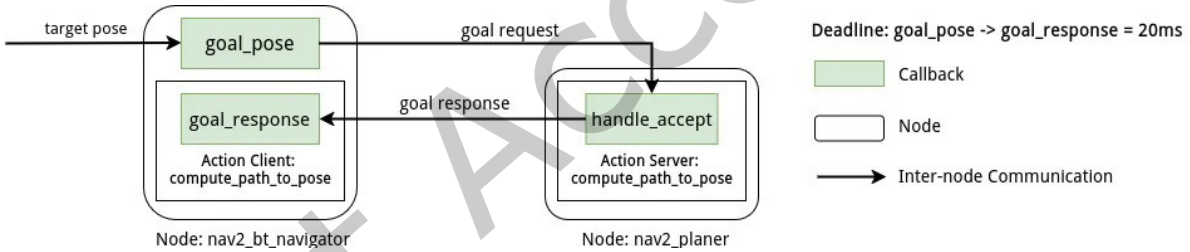


Fig. 4. Example of a partial cause-effect chain in Navigation2. The chain begins when a target position is received by the `goal_pose` subscriber, triggering the first callback. The target is then forwarded to the `Action Server: compute_path_to_pose` in the global planner. Upon accepting the goal, the Action Server sends a goal response to the corresponding Action Client in the BT Navigator, where it is processed by the `goal_response` callback.

However, through our empirical study of real-world ROS2 applications, we observe that partial cause-effect chains are equally critical and also subject to hard real-time constraints. Unlike end-to-end chains, the starting callback of a ***partial chain*** is not necessarily a timer callback, and it may not extend all the way to actuators. One prominent example is found in the widely adopted Navigation2 stack, illustrated in Fig. 4. Within Navigation2, a partial cause-effect chain starts from the reception of a navigation goal at `goal_pose` (a subscriber callback) and ends at the invocation of `goal_response` (a client callback) in `compute_path_to_pose`, an Action Client responsible for global path planning. This chain does not directly involve actuator commands, but is a critical step in achieving autonomous navigation behavior.

Navigation2 enforces a *hardcoded* deadline of 20 ms by default on this chain: from the moment the `goal_pose` receives the target pose, the Action Client `compute_path_to_pose`'s `goal_response` callback must receive

Just Accepted

the goal response within 20 ms. If this constraint is violated, Navigation2 automatically invokes its ***Recovery mechanism***, which attempts to restore functionality by performing a series of fallback operations such as clearing global or local costmaps, spinning, waiting, or backing up, as depicted in Fig. 5.

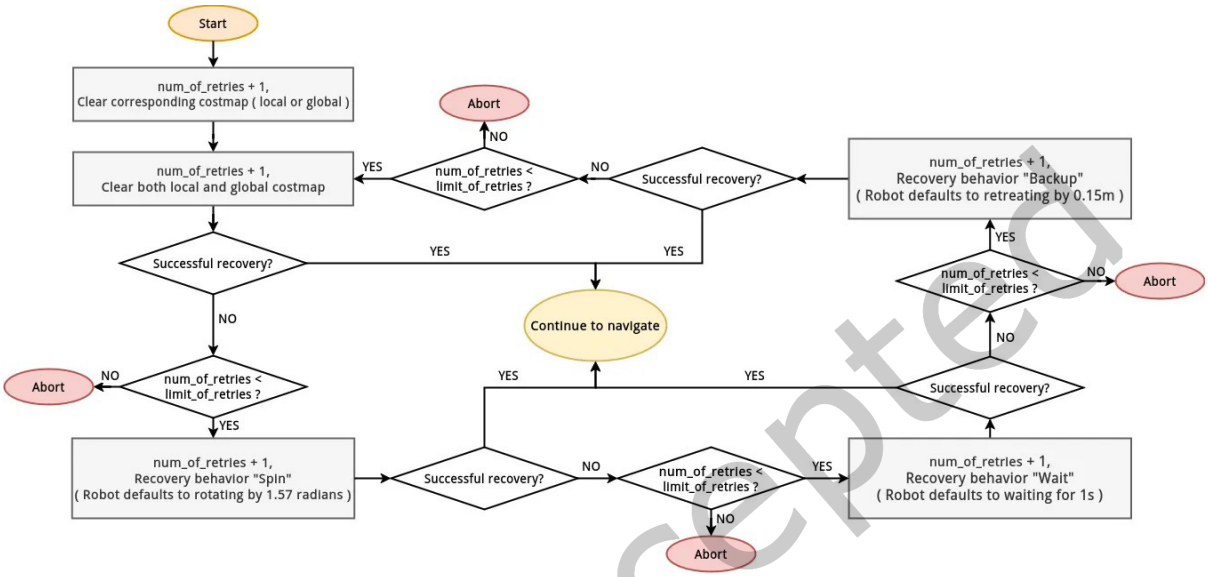


Fig. 5. Default operation sequence of Navigation2's Recovery mechanism. Upon failure, the system clears the relevant costmap and increments `num_of_retries` count. If the retry count is below `limit_of_retries`, it retries the failed functionality; on success, normal operation resumes. Otherwise, a sequence of recovery behaviors, including spin, wait, backup, and clear local and global costmap, is executed after each failed retry. This loop continues until recovery succeeds or the retry limit is reached.

Just Accepted

While Recovery mechanism is essential for handling rare runtime failures, triggering it during normal operation can have severe consequences on the system's functional performance–for example, reducing navigation speed or introducing oscillatory behavior. More importantly, because the Recovery operations vary based on the number of consecutive invocations, the system's behavior becomes unpredictable once Recovery is triggered. As a result, Recovery is not deemed as part of the normal execution path, and avoiding its invocation is crucial for maintaining system stability and performance. In Section 6, we will show an experimental evaluation on the impact of missing the deadline for this partical chain on the functional performance of an autonomous robot.

In this context, the 20 ms deadline imposed on this partial cause-effect chain represents a hard real-time constraint, even though the chain does not span from sensor to actuator. This highlights the need to expand the scope of real-time analysis in ROS2 systems to include partial cause-effect chains, especially those that govern critical internal decision-making processes such as navigation.

Since partial chains are often not triggered periodically by sensors, their response time analysis must account for sporadic release patterns, as considered in works such as [21–23]. To ensure these partial chains meet their deadlines, response times can be improved through deliberate assignment and scheduling of the associated callbacks. Furthermore, explicit modeling of the partial chains' release patterns due to preceding callbacks, rather than relying solely on worst-case behaviors, can also make the response time analysis tighter.

4.2 Runtime Variability in Structure and Timing of Cause-Effect Chains

In real-world ROS2-based autonomous applications, cause-effect chains do not always exhibit fixed, static structures. Instead, both the structure and timing characteristics of these chains can change dynamically at runtime, depending on the system's internal or external state. This runtime variability poses challenges to worst-case response time (WCRT) analysis approaches, which often assume execution chains with fixed structures and near-constant execution times for callbacks.

A representative example is found in Navigation2, which was briefly introducted in Section 3.4. Navigation2 separates navigation into two major phases: Global Planning and Local Planning. Global Planning computes a path to the goal on a static map, and by default uses the NavFn planner plugin with Dijkstra's expansion (assuming a 2D holonomic robot model). Local Planning, on the other hand, is responsible for generating real-time control commands to follow the global path while avoiding obstacles, using the Dynamic Window Approach-based controller by default.

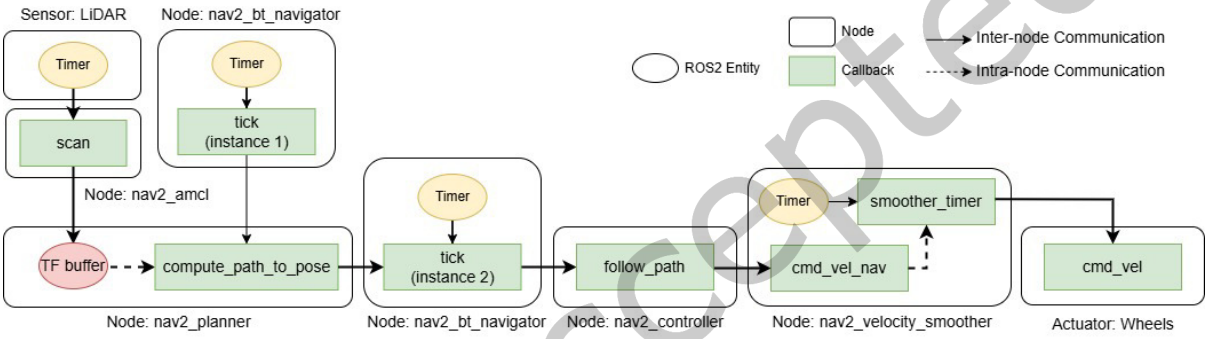


Fig. 6. Structure of the cause-effect chain from LiDAR to wheels during the global planning state of Navigation2. The chain flows from LiDAR data through the `nav2_planner` node (Global Planner), activated by the `nav2_bt_navigator` node via managing the Behavior Tree's ticks, then proceeds to the `nav2_controller` node (Local Planner) and velocity smoother, and finally generates velocity commands sent to the actuator for wheels. This chain represents the full planning and control pipeline executed during the first navigation cycle.

In Navigation2-based systems, the end-to-end cause-effect chain from LiDAR to wheels exhibits dynamically changing structure. As shown in Fig. 6, when the position of a new navigation goal is first received, the system initiates a global plan: LiDAR data is stored in a TF Buffer and then retrieved by the `nav2_planner` node (Global Planner) when a Behavior Tree controlled by the `nav2_bt_navigator` node ticks. The Behavior Tree ticks at a fixed rate and governs transitions between planning and control states. In the first tick, the tree activates Global Planning by activating `nav2_planner` to compute path to goal position. In the subsequent tick, the system transitions into a cruising state and activates the Local Planning phase by sending the path to the `nav2_controller` node, which through a series of processing computes velocity commands for the wheels.

In the cruising state, the Global Planner is activated only once per second by default. Between activations, LiDAR data bypasses both `nav2_planner` and `nav2_bt_navigator`, flowing directly from the sensor to the Local Planner and then to the actuator, as shown in Fig. 7. This illustrates that even when the source and destination callbacks remain fixed, the internal structure of the cause-effect chain can vary dynamically at runtime depending on the system state.

This observation reveals a limitation in existing real-time analysis approaches for ROS2 that model individual static worst-case chains. Using the longest structural path (i.e., including Global Planning) to model the worst-case

Just Accepted

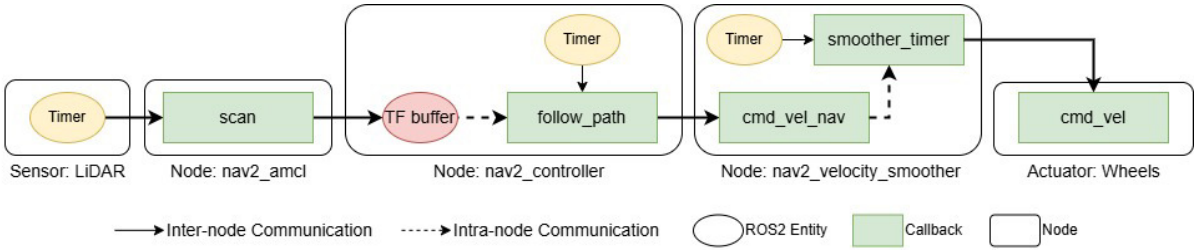


Fig. 7. Structure of the cause-effect chain from LiDAR to wheels during the cruising state when Global Planner is not activated. The `nav2_amcl` node receives LiDAR data via the Scan callback and computes the robot's pose, which is stored in the TF buffer of the `nav2_controller` node. Its `follow_path` callback, triggered by a timer, accesses this pose through the TF buffer and computes velocity, which are published and received by the `cmd_vel_nav` subscriber callback. The computed velocity is stored internally and later accessed by the `smoother_timer` callback, which performs smoothing and sends the final command to the actuator.

behavior of all scenarios would result in overly pessimistic WCRT during most of the system's runtime. While one could attempt to apply multi-mode analysis techniques from classical real-time systems theory to model these variations, the potentially high frequency of mode transitions in ROS2 applications–especially within Navigation2 where Global Planner is activated once every second–introduces additional complexity. In particular, ensuring real-time guarantees during the mode transitions becomes a non-trivial problem.

Similarly, cause-effect chains involved in Navigation2's Recovery mechanism, introduced in Section 4.1, also exhibit dynamic structures. The behavior during recovery is state-dependent: it may involve clearing global/local costmaps, spinning in place, backing up, or pausing execution. If clearing costmap is sufficient for recovery, the system continues executing other navigation modules in parallel with recovery; in other cases, the recovery mechanism may reset the entire navigation stack. This structural variability complicates the timing analysis.

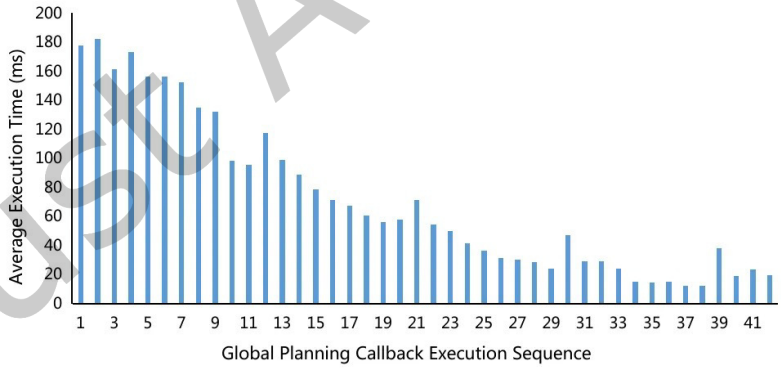


Fig. 8. Execution time of the Global Planning callback decreases as the robot approaches the target goal. The measurement, taken in a warehouse scenario using Navigation2 with the default Dijkstra-based planner, shows a gradual reduction in execution time from approximately 180 ms at the start of navigation to less than 20 ms near the goal position.

In addition to structural variability, real-world cause-effect chains also exhibit dynamic execution time behavior, as pointed out by authors of ROS-Llama [7]. Certain callbacks have execution times that change significantly depending on the system's state or environment. For instance, in a warehouse scenario using a TurtleBot3 Waffle

Just Accepted

robot running Navigation2, we observed this effect in the Global Planning callback. With Dijkstra's expansion as the default algorithm, the planner is triggered every second. During a run from position $(0, 0)$ to $(7.78, -7.00)$, we recorded the execution time of the planner over time in Fig. 8. At the start of navigation, the planner took approximately 180 ms to compute a path; however, as the robot approached the goal, execution time gradually dropped to as low as 20 ms. If one models the Global Planner's callback using a conservative WCET of 180 ms, then the WCRT of any cause-effect chain containing that callback will be overly pessimistic throughout a major of the system's operation.

In summary, both structural and timing variability are intrinsic to real-world ROS2 applications. Accurately capturing and analyzing these characteristics is essential for deriving less pessimistic real-time guarantees. Future work should explore dynamic models of chain composition and execution behavior, as well as analysis techniques that can handle frequent and state-dependent changes in chain configuration. The analysis can integrate real-time information obtained through online monitoring and adaptive scheduling strategies in response to variability at runtime, to enable tighter bounds and better performance in dynamic environments.

4.3 Non-FIFO Data Access Patterns in ROS2 Communication

Existing studies on timing analysis for ROS2 assume that the data buffers or message queues used for communication operate in FIFO order with either unbounded sizes or bounded sizes incorporating data refreshing semantics. However, in real-world ROS2 applications, this assumption does not always hold. Due to both application-level design choices and characteristics of the ROS2 communication mechanism, downstream callbacks may deliberately or implicitly operate on old data with potentially non-FIFO ordering, which alters the system's timing and functional behavior.

Application-Level Use of Data with Non-FIFO Ordering. In some applications, downstream callbacks are intentionally designed to use past data produced by upstream components in Non-FIFO ordering to satisfy consistency or functional requirements. A representative example arises in Navigation2 during the interaction between Global Planning and Local Planning.

In Navigation2, the TF buffer plays a critical role in enabling spatial transformations between different coordinate frames. The TF buffer is updated by multiple sensors, with each sensor contributing data to different frames. For example, the `map` frame is updated by LiDAR, while the `odom` frame is updated by wheel odometry. In addition to the different update rates of sensors, due to the calculation and update of AMCL in Navigation2, the data from the LiDAR (for the `map` frame) is always about 1 second recent than the odometry data (for the `odom` frame) in the TF buffer.

During local planning, it is necessary to transform the robot's pose from the `odom` frame into the `map` frame. Due to the difference in update frequencies and timestamps, a consistency issue arises: the latest available odometry data typically lags behind the latest LiDAR data in time. To maintain temporal consistency in such transformations, the local planner must query not only the most recent LiDAR data but also the LiDAR data that corresponds to the timestamp of the odometry. This means the system must simultaneously access both new and old LiDAR data from the TF buffer during a single planning cycle. As illustrated in Fig. 9, the `follow_path` callback in the `nav2_controller` node (responsible for local planning), specifically the callback instance labeled f_3, accesses two LiDAR data points: the most recent one at time t_3, and an older one at time t_1 that aligns with the timestamp of the odometry input. Notably, the data from t_1 was already used by a prior callback instance f_1, demonstrating a reuse of older data that violates the FIFO semantics typically assumed in timing analysis. In addition to reading one old data and one new data like `follow_path`, there is a callback in Global Planning where two old data are read.

Such behavior invalidates the assumption on ROS2 timing analysis that data is consumed in FIFO order, where newer data always supersedes older data. A similar example is the PointPainting node in Autoware's

Just Accepted

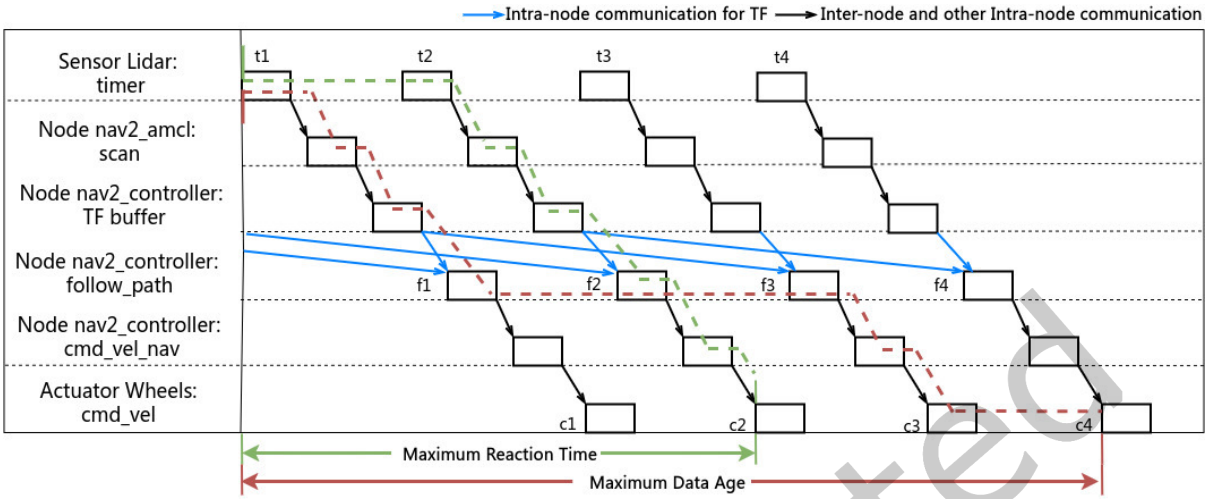


Fig. 9. Instances of cause-effect chain from sensor LiDAR to actuator wheels. To ensure temporal consistency, each instance of callback `follow_path` explicitly uses both the most recent and the past LiDAR data from TB buffer, resulting in different Maximum Reaction Time and Maximum Data Age.

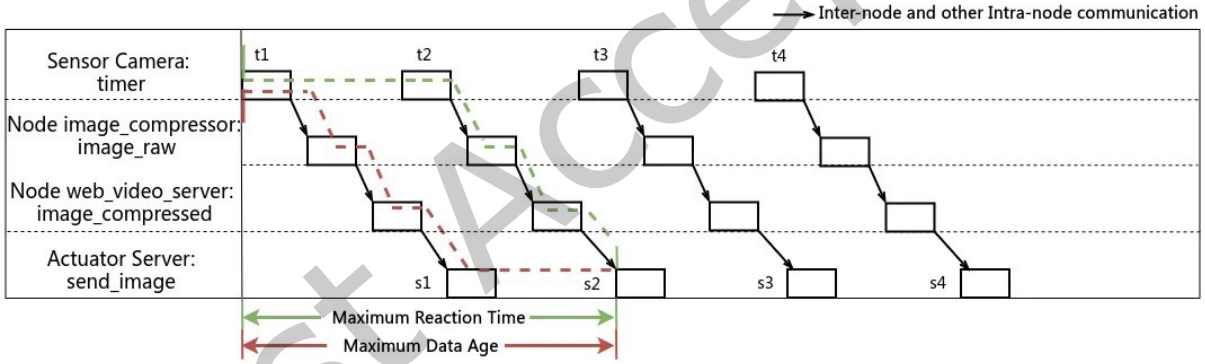


Fig. 10. Instances of the cause-effect chain in Fig. 13 where all callbacks only read the most recent data. In contrast to Fig. 9, these instances of cause-effect only involve data access in FIFO order, resulting in the Maximum Reaction Time being equal to the Maximum Data Age.

Just Accepted

`autoware_image_projection_based_fusion` module, which performs fusion between point clouds and RGB images. To ensure consistency, the transformation uses TF data corresponding to the timestamp of the point cloud, which may not be the latest available data.

As described in Section 3.3, Maximum Reaction Time is the maximum latency between the occurrence of an external event and the first time it impacts the system's behavior, while Maximum Data Age captures how long a sampled event continues to influence the system. When only using the newest data, Maximum Data Age and Maximum Reaction Time are equal, as shown in Fig. 10.

However, when older data can be used, this property may no longer hold. As in Fig. 9, an external event occurs slightly after t_1, is first detected by the LiDAR at t_2, and takes effect through a downstream callback c_2. The green

dashed line highlights the corresponding Maximum Reaction Time. Meanwhile, the LiDAR data captured at t_1 is lastly used by callback f_3, taking effect through callback c_3, and remains effective until callback c_4 uses newer data. This persistence of influence is marked by the red dashed line, representing the Maximum Data Age.

Clearly, when data reuse or out-of-order access occurs, these two metrics–Maximum Data Age may exceed Maximum Reaction Time–no longer align. Consequently, timing analysis frameworks must be extended or revised to accurately capture the effects of non-FIFO data usage in real-world ROS2 applications. The frameworks should explicitly identify data usage patterns within callbacks to determine which older data instances may be consumed, and developing worst-case analysis techniques that bound the age of such data under varying scheduling and execution conditions.

Functional Performance Impacted by ROS2's Bounded Queues. As discussed in [23, 24], ROS2's default communication semantics can also lead to downstream callbacks processing stale messages. Specifically, entities such as subscribers, clients, and services maintain a message queue with a default size of 10, which can be configured by the developer. When an upstream publisher pushes messages at a higher rate than a downstream callback can process, the queue accumulates messages. Each time the callback is triggered, it processes the oldest message at the front of the queue, not necessarily the latest available message. If the queue becomes full, ROS2 drops the oldest message upon enqueueing a new one.

In addition to the impact on timing analysis, we want to highlight here that this behavior also impacts the functional behavior of real-world applications. In particular, message loss may occur when the queue overflows, breaking parts of the cause-effect chain and leading to gaps in data propagation from sensors to actuators.

To empirically evaluate this issue, we designed a test case using the TurtleBot3 Waffle robot in a Gazebo simulation environment. The robot runs Navigation2 as its navigation stack and is equipped with a camera-based pedestrian detector using YOLOv3 Tiny. We placed a stationary human in the robot's path and instructed the robot to stop immediately upon detection. YOLOv3 Tiny was isolated on a dedicated CPU core, achieving an average inference time of about 101.6 ms.

Table 1. Average distance in front of the human after the robot comes to a full stop with various buffer sizes of message queues and frequency of the camera.

Buffer Size	Camera Frame Rate	Average Reaction Time (ms)	Sensor Data Loss Ratio	Average Stopping Distance (larger is better)
10	60hz	622.3	90.27%	2.67
10	20hz	974.2	78.24%	2.56
10	10hz	1322.9	55.17%	2.48
10	4hz	381.7	0.6%	2.77
1	60hz	430.4	90.84%	2.64
1	20hz	447.0	75.97%	2.69
1	10hz	484.64	56.34%	2.70
1	4hz	403.6	1.73%	2.64

We varied the camera frame rate (4, 10, 20, and 60 Hz) and the ROS2 subscriber buffer size (1 and 10), then recorded the robot's average stopping distance across 10 trials for each configuration. Results in Table 1 show that when the camera frequency exceeded 4 Hz, the core running YOLOv3 inference started to become more than saturated. On one hand, the message queue was quickly filled, causing higher sensor data loss ratios. With a buffer size of 10 and a camera frame rate higher than 4 Hz, YOLOv3 consistently processed outdated frames–up to 10 frames old–rather than the most recent frame. This caused the robot to brake later than expected, decreasing the stopping distance. On the other hand, with higher data loss ratios, older sensor data were dropped out of the

Just Accepted

buffer faster, so newer (yet not the newest) sensor data are used by YOLOv3 inference. In comparison, with a buffer size of 1, the newest sensor data is always used, so there was not a significant decrease of stopping distance with 10 Hz frequency.

When buffer overflow occurs, the relationship between reaction time and functional performance becomes more complex and less predictable. Although core overloading and queue overflow should typically be avoided in practice, these findings highlight the importance of studying how bounded queues impact both real-time guarantees and functional correctness. Future research should explore analytical models that characterize this impact under varying system loads and scheduling strategies. Furthermore, when the goal is to optimize functional performance, one potential research direction is to determine the optimal queue size analytically, and then introduce explicit real-time constraints or admission control mechanisms to prevent queue overflow while preserving task schedulability.

4.4 Executor May Not Control All Computation Threads in ROS2 Applications

While ROS2 provides a well-defined execution framework based on the concept of executors managing and scheduling callbacks, not all computational components in real-world applications are executed under the control of ROS2 executors. In fact, several critical components are explicitly implemented outside the executor, running in manually created threads managed directly through operating system APIs. These unmanaged threads can consume large computational resources and play a crucial role in the functionality and timing behavior of the system.

A prominent example arises in the execution of long-running components using the ROS2 Action interface. Actions are commonly used for operations that require substantial time to complete, such as Global Planning in Navigation2. Since the ROS2 executor performs non-preemptive scheduling, long-running components can block the execution of other callbacks if they share the same executor thread. To avoid this, application developers typically move the execution of long components into separate threads, created via direct calls to the operating system (e.g., `std::thread` or `pthread`). These threads are spawned inside the Action Server, where the actual component logic is executed. The Action Client sends a goal request, and upon acceptance, the Action Server creates a new thread to execute the component. Once the component is complete, the thread is destroyed.

This design pattern–offloading computation to threads unmanaged by ROS2–is not limited to actions. Many core modules in mainstream robotic frameworks adopt similar strategies. For instance, in Navigation2, the Costmap module relies on a background thread to update its internal 2D occupancy grid. Specifically, the `Costmap2DROS::mapUpdateLoop` function is launched during node initialization and continuously runs until the system shuts down. This loop periodically updates the costmap based on sensor data, with a default frequency of 1 Hz for the global costmap and 5 Hz for the local costmap. Another example is the `ndt_scan_matcher` node in Autoware, which can utilize `ndt_omp`–a variant of the Normal Distributions Tansform (NDT) algorithm implemented with OpenMP. Many critical functions within this module employ `#pragma omp parallel for` to spawn multiple OpenMP threads for parallel execution. These long-lived threads are not managed or scheduled by the ROS2 Executor — they are entirely managed by the application and OS. Consequently, they compete for CPU resources independently and may introduce additional timing variability and interference that standard ROS2 scheduling analyses may not capture.

Despite being outside the ROS2 executor's control, these uncontrolled components are integral parts of the application's cause-effect chains. For example, a chain from LiDAR input to actuator output may include a costmap update, a local planner computation, and a velocity publisher–all working across both controlled and uncontrolled components. Therefore, analyzing the timing behavior of a real-world ROS2 system must account for these non-executor-controlled threads. Ignoring them would lead to incomplete or inaccurate characterizations of latency and schedulability.

Just Accepted

In summary, real-world ROS2 applications rely on a combination of executor-controlled callbacks and manually managed threads. These uncontrolled computation threads participate in the same cause-effect chains as ROS-native callbacks and must be considered as first-class entities in real-time analysis. Future research should aim to systematically model these threads within the real-time scheduling framework, accounting for their interaction patterns, CPU contention, and state-dependent activation. Additionally, developing standardized annotations or interfaces to expose thread behaviors to the ROS2 runtime or even be incorporated into the executor scheduling algorithm could enable more accurate and integrated scheduling analyses.

5 Extending the Tracing Toolchain for TF and Action Support in ROS2

To analyze the timing and structural properties of cause-effect chains in ROS2 applications, we extend the open-source tracing and analysis tool `ros2_tracing` originally developed by C. Bédard et al. [4]. Their tool instruments ROS2 with tracepoints–already integrated into the mainline ROS2 codebase–and uses the `LTTng` tracing framework to record runtime event information into trace files. Since the concept of a cause-effect chain is logical and not explicitly defined within the ROS2 system, the tool needs to reconstruct cause-effect chains using tracepoints. It originally supports cause-effect chains composed of Timer, Publisher, and Subscriber entities and visualize these chains through a GUI interface. We build upon this foundation by adding support for TF-based and Action-based cause-effect chains. Additionally, we implement features for batch exporting cause-effect chains and calculating metrics such as response time of end-to-end or partial chains. This enables automated analysis using scripts to extract runtime statistics across multiple chain instances.

5.1 Integrating TF for Cause-Effect Chain Tracing

To enable tracing of TF-related data dependencies, we introduce new tracepoints specifically designed to track how TF buffers are updated and queried during coordinate transformations.

In ROS2, TF data is accessed through per-node TF buffer structures, implementing a distributed buffer design. TF buffers store time-stamped transform messages that originate from different sensors. For example, in Navigation2, the AMCL node processes LiDAR data and periodically updates the transform from the `odom` frame to the `map` frame. These updates are stored in the TF buffer for use by downstream callbacks. When a callback such as `follow_path` performs a transformation (e.g., from odom to map), it queries the buffer using the relevant timestamp and iteratively traverses the TF tree to compute the transformation. This forms a case of intra-node communication, which cannot be captured by analyzing Publisher–Subscriber links alone supported in `ros2_tracing`. To trace this behavior, we introduce three new tracepoints:

- `store_in_buffer`: Records the storage of a transform message m in the TF buffer at time t, along with the preceding callback instance c_t that generated it. This callback instance c_t is the tail of a upstream partial chain.
- `calculate_in_buffer`: Logs the usage of a stored message m' in a coordinate transformation at time t', along with the succeeding callback instance $c_{t'}$ that performed the calculation using the message. This callback instance $c_{t'}$ is the head of a downstream partial chain.
- `interpolate_in_buffer`: Similar to `calculate_in_buffer`, but used when two TF messages m' and m'' are interpolated to compute a transform. This tracepoint is associated with the callback instance performing the computation, which is also the head of a downstream partial chain.

The tool has been modified to parse these new tracepoints from the trace logs and infer connections between upstream and downstream partial chain instances. When a `calculate_in_buffer` or `interpolate_in_buffer` uses a message previously recorded by `store_in_buffer`, the tool links the corresponding callbacks to form a complete cause-effect chain segment that includes TF-mediated dependencies.

Just Accepted

5.2 Integrating Action for Cause-Effect Chain Tracing

The Action interface in ROS2 encapsulates a multi-phase interaction pattern involving long-running computations, and is composed of four sub-entities: Client, Service, Publisher, and Subscriber. To support actions in the tool, we first extended the tracing framework to recognize Client–Service communication, which differs from Publisher–Subscriber in that it is bidirectional. Specifically, the tool now tracks both the request from client to service and the response from service to client, using newly added tracepoints that record the identifiers and timestamps of each message exchange.

Furthermore, in real-world applications such as Navigation2, long-running tasks associated with actions (e.g., global path planning) are typically offloaded to separate threads created via OS-level APIs. These threads are not managed by the ROS2 executor, and thus cannot be automatically traced without additional instrumentation. To address this, we introduce two new tracepoints: (1) `action_callback_start` placed at the beginning of the long-running computation, and (2) `action_callback_end` placed at the end of the computation.

By inserting these tracepoints into the application source code, we enable the tracing tool to detect and measure the execution duration of long-running actions. These tracepoints allow the tool to construct action-related cause-effect chains that span the Action Client, Server, and their respective thread-based callbacks, capturing the full temporal footprint of the action and providing insight into its contribution to end-to-end latency.

5.3 Tracing Overhead

The tracing overhead introduced by our extended tool is minimal. As discussed above, our extension inherits the tracing infrastructure of the official `ros2_tracing` package, with additional tracepoints inserted into the ROS2 execution path. As discussed in the original paper for `ros2_tracing` [4], tracing overhead depends on the data recorded per tracepoint. Existing tracepoints incur an average overhead of approximately 0.003 ms for tracepoints writing data ranging from 1–256 KB. The additional tracepoints we introduced write at most 48 bytes of data, and our measurements show an average overhead of just 0.0012 ms. Thus, our tracing extension maintains low runtime impact while providing additional tracing capabilities for TF and Action.

6 Experiments in Simulated Autonomous Robotic Scenarios

To empirically validate the observations presented in Section 4, we constructed a simulated autonomous robotic environment using Gazebo and the TurtleBot3 Waffle platform. In this environment, we developed two representative application scenarios that explicitly exhibit the structural and behavioral characteristics of cause-effect chains discussed earlier. Alongside the application scenarios, we experimented with a set of scheduling configurations. These configurations allow us to observe how scheduling decisions affect key real-time performance metrics and how those timing behaviors, in turn, impact the functional performance of the robotic system. These experiments demonstrate that the observations made in Section 4 are not only theoretically significant but also practically relevant to the design and operation of real-world robotic and autonomous systems.

6.1 Experimental Setup

The experiments were conducted using two separate server machines: one dedicated to running the Gazebo simulator and the other serving as the experimental system that executed the ROS2-based applications, so that the simulation does not affect the behavior of the ROS2-based applications, mimicking the real-world execution. As shown in Fig. 11, the two machines were directly connected via an Ethernet cable with a bandwidth of 1000 Mb/s to ensure low-latency and high-throughput communication. The simulation environment was built on Gazebo 11.10.2, and both machines ran ROS2 Humble to ensure consistency in the runtime environment.

The experimental machine was configured with Linux kernel 6.8.2 patched with real-time patch and was based on Ubuntu 22.04.2. It featured an AMD Ryzen 5 5600G CPU with hyper-threading disabled. Core isolation

Just Accepted

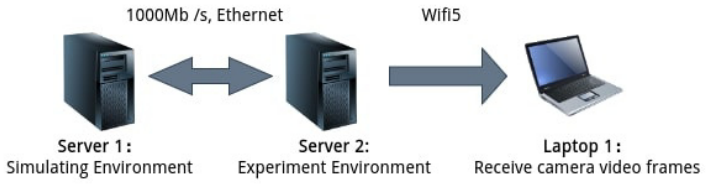


Fig. 11. Network configuration used in the experimental setup. Server 1 (running Gazebo) and Server 2 (running the ROS2 application) are directly connected via Ethernet with a bandwidth of 1000 Mb/s, exceeding the communication demands of the simulation. Laptop 1 connects to Server 2 via Wi-Fi 5 on a separate subnet, ensuring no interference with the wired connection between the servers.

was configured with `isolcpus=5,4,3,2,1`, reserving core 0 for the operating system and isolating cores 1 to 5 exclusively for ROS2 application processes. To ensure timing determinism, the CPU frequency was fixed at 1.4 GHz. The system had 16 GB of RAM to provide sufficient memory resources during testing.

In the simulation, the robot model was TurtleBot3 Waffle, and a virtual RGB camera was used with a resolution of 480×320. The simulated camera streamed images encoded in BGR8 format.

In the first application scenario, the robot needs to transmit the camera stream to other devices over the Wi-Fi network for further storing and processing. In this setup, the camera stream was compressed and served via the open-source `web_video_server`, which allowed other devices to access the video stream over the network. A separate laptop connected via Wi-Fi was used to receive and monitor this stream in real time, as shown in Fig. 11.

For object detection, we employed a YOLOv3 Tiny model, which provided lightweight yet effective visual perception capabilities suitable for real-time robotic applications. The frequency of Global Planning in the Navigation2 stack was set to 6 Hz, matching typical autonomous navigation update rates. In each experiment, we run each different configuration for 10 times to ensure statistical robustness of the results.

6.2 Scheduling Configurations

To evaluate the impact of different scheduling strategies on ROS2 applications, we designed a set of configurations. These configurations specify how ROS2 nodes are partitioned across CPU cores and which OS-level scheduling policies are applied. While the setups are simple, deliberate, and not optimized for performance, they are constructed to highlight the runtime behaviors discussed in Section 4 and to demonstrate how scheduling decisions influence both the real-time responsiveness and functional performance of an autonomous system.

Redundant Configuration: Designed to provide each application component with sufficient system resources, ensuring that all real-time constraints and responsiveness requirements are met with minimal interference. To achieve this, CPU cores are deliberately under-utilized. In order to avoid non-determinism caused by the operating system migrating threads across cores during repeated trials, ROS2 nodes assigned to dedicated executors whose threads are pinned to fixed cores, as detailed in Table 2. To ensure fair execution among multiple executor threads on the same core, we use the Completely Fair Scheduler (CFS) instead of enforcing static priorities. This configuration serves as a best-effort baseline, minimizing non-determinism from scheduling or resource constraints. In each scenario, it is expected to exhibit the (near) best real-time and functional performance, serving as a reference and highlighting the performance degradation or timing variations introduced by more constrained resources or imbalanced setups.

Configuration 1: Modified from the Redundant Configuration by introducing a deliberate resource contention scenario to study the impact of ROS2 executor scheduling on real-time performance and functional behavior. Specifically, we focus on the `nav2_amcl` node, which handles robot localization. Its core callback, `scan`, is

Just Accepted

Table 2. Redundant Configuration

Node Name	Node Functionality	Assigned CPU Core
cv_detection	object tracker based on YOLOv3 tiny	5
robot_planner	strategy (decision-making)	4
web_video_server	streaming of camera video	4
camera_img_recorder	record camera image to disk	3
nav2_map_server	map management	1
nav2_amcl	localization	1
nav2_lifecycle_manager	manage lifecycle nodes in the localization module	1
nav2_controller	local planning	2
nav2_smoother	path smoothing	2
nav2_planner	global planning	3
nav2_behaviors	task server for executing behaviors (e.g. Spin, Backup)	3
nav2_bt_navigator	receive goal pose & manage the behavior tree	3
nav2_velocity_smoother	velocity smoothing	4
nav2_lifecycle_manager	manage lifecycle nodes in the navigation module	4

responsible for subscribing to LiDAR data on the scan topic and performing particle filter-based pose estimation–one of the most critical tasks for localization. In this configuration, we reassign the `nav2_amcl` node to core 5, where it shares computational resources with the `cv_detection` node, an object tracking module based on visual input. We bind both `nav2_amcl`'s `scan` callback and the `cv_detection` node to the same ROS2 executor, which runs on core 5. In this setup, these callbacks are scheduled using ROS2 executor's scheduler and run non-preemptively. Importantly, no other executors or threads are placed on core 5. All other settings follow those of the Redundant Configuration.

Configuration 2: Extended from the Redundant Configuration by introducing both inter-executor contention and preemption between threads. In this setup, the `nav2_planner` node, which implements the Global Planning functionality using the ROS2 Action interface, is split across cores. Specifically, its Action Server callback, responsible for handling incoming planning goals, is moved to core 5, while the remaining callbacks and the long-running planning thread remain on core 3. Additionally, the `camera_img_recorder` node, which continuously logs camera frames, is also reassigned to core 5. The `camera_img_recorder` and the Action Server callback from `nav2_planner` are bound to the same ROS2 executor, which itself is pinned to core 5. This creates potential executor-level scheduling contention between the two callbacks. Meanwhile, the `cv_detection` node in a separate thread, originally designed to run with exclusive access in the Redundant Configuration, now also contend with the other executor thread on core 5. To predictably examine the impact of thread-level preemptions, we assign the executor thread that hosts `camera_img_recorder` and Action Server callback a real-time priority of 5, which is higher than that of `cv_detection`. All other settings are the same as the Redundant Configuration.

Configuration 3: Designed to investigate the impact of threads not controlled by ROS2 on the timing of cause-effect chains. Building on the Redundant Configuration, this setup modifies the placement of the Global Planning module in the `nav2_planner` node–specifically, the long-running planning thread that is spawned by the Action Server but executed outside the control of the ROS2 executor. This planning thread is pinned to core 5. In addition, the `web_video_server` node, responsible for streaming compressed camera video over the network, is also moved to core 5. Unlike the planning thread, this node is managed by a ROS2 executor. To

Just Accepted

reduce variability from unrelated contention, the `cv_detection` node–previously located on core 5–is reassigned to core 4, the core with the lowest utilization. Thus, on core 5, only two threads are active: the long-running planning thread and the executor thread managing `web_video_server`. We assign these threads priorities of 5 and 3, respectively, giving the global planning thread higher priority.

6.3 Application Scenario 1: Point-to-Point Navigation with Video Streaming

In this scenario, the robot operates within a warehouse environment and performs a simple point-to-point navigation task. Specifically, the robot is instructed to move from position (0,0) to (3.72, -8.62) while simultaneously streaming video from its onboard camera. The camera feed is transmitted over the local network, and Laptop 1 receives the video stream in real time. This scenario is intentionally designed as a single-goal task for evaluating the performance of different system configurations with a relatively simple functional goal. It allows us to observe how scheduling and resource allocation strategies affect both real-time performance metrics and the functional reliability of the navigation and video streaming processes.

Cause-Effect Chains under Consideration. A total of five cause-effect chains are considered in this application scenario: Chains 1 to 4 illustrated in Fig. 12 represent important partial chains, while Chain 5 in Fig. 13 is a scenario-specific end-to-end chain corresponding to camera streaming.

Chain 1 and Chain 2 capture the flow of LiDAR data through the localization module `nav2_amcl`, after which the localization results are respectively passed to the Local Planning and Global Planning pipelines. In Chain 1, LiDAR samples are used to update the robot's pose estimate, which is then consumed by the Local Planning module to compute motion commands. The real-time performance of this chain reflects the latency between perception-based localization and control execution. The deadline for Chain 1 is determined based on Navigation2's default setting. Specifically, it is the sum of two parameters: the `transform_tolerance` value in the AMCL module (default 1.0 s), and the `transform_tolerance` in the Controller Server (Local Planning) module (default 0.2 s), totaling 1.2 seconds. This deadline represents the maximum acceptable delay from the timestamp of a LiDAR data to the execution of the `follow_path` callback in the Controller Server. If the maximum reaction time of Chain 1 exceeds 1.2 seconds, the `follow_path` callback will reject the pose data as too stale, resulting in failure to compute a valid motion command and causing the robot to receive a zero-velocity output.

In Chain 2, the updated localization is used to inform Global Planning, whose computed path is eventually passed downstream to Local Planning. This chain captures the timing behavior of localization results flowing through the entire planning stack before influencing actuation.

Chain 3 and Chain 4 represent how LiDAR data contributes to environmental awareness via the Costmap modules. In Chain 3, LiDAR observations are transformed using the TF buffer and incorporated into the Global Costmap, which is then accessed by Global Planning. This chain's timing behavior reflects the system's ability to detect obstacles quick for further avoidance. Chain 4 follows a similar pattern but updates the Local Costmap, which is directly consumed by Local Planning for short-term obstacle avoidance. Both chains rely on TF-based coordinate transformation and map updates, and their real-time performance captures the reaction latency from sensing an obstacle to adapting navigation behavior.

The deadlines for Chains 2-4 are determined by the maximum time the robot can take to stop safely upon detecting an obstacle. In the warehouse environment, the minimum spacing between shelves is 2 meters. Assuming the robot travels at its maximum speed directly toward a shelf, the maximum allowable reaction time before collision is approximately 3.846 seconds, which is used as the deadline for these chains to ensure timely obstacle detection and avoidance.

In addition to these four chains, Chain 5 in Fig. 13 is used to evaluate application-specific functionality: the real-time streaming of camera data. This end-to-end chain begins with image capture on the robot and ends with video delivery by the `web_video_server`. It represents the perceptual pipeline supporting external monitoring

Just Accepted

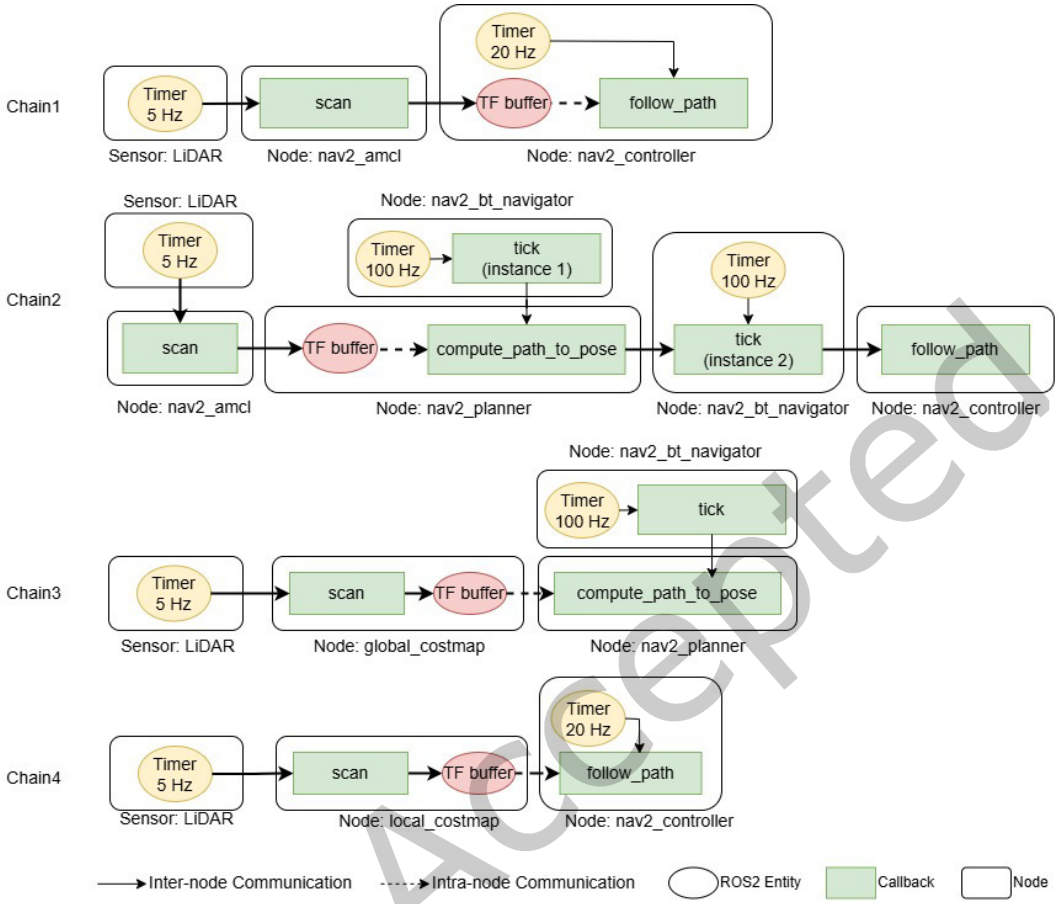


Fig. 12. Structure of Chains 1-4. Chain 1: LiDAR data is received by `nav2_amcl`'s `scan` callback to compute the robot's pose in the map frame, which is later retrieved via TF buffer by `follow_path`. Chain 2: Similar to Chain 1, but the pose is used by `compute_path_to_pose` for global planning and then passed to `follow_path` through the `nav2_bt_navigator`. Chain 3: LiDAR data is processed to update the Global Costmap with info like obstacles, which is then read by `compute_path_to_pose` via TF buffer. Chain 4: LiDAR data is used by the `local_costmap` and consumed by `follow_path` via TF buffer for local obstacle avoidance.

Just Accepted

or visual-based feedback. The `image_compressor` node for raw data is on Server 1 (simulation machine), while streaming is handled on Server 2. The deadline for Chain 5 is 20 ms, determined by the `bt_navigator` node's default `default_server_timeout` parameter. During execution, the behavior tree checks whether the Action Client receives acknowledgment from the Action Server within this timeout. Exceeding this limit constitutes a deadline miss, triggering recovery behavior.

For these five chains, we evaluate the following real-time performance metrics: (1) Maximum Reaction Time of a partial or end-to-end chain. (2) Missed Deadlines, specifically for Chain 1. This partial chain from LiDAR to `follow_path` has a default deadline of 1.2 seconds in Navigation2. If this deadline is violated, Local Planning outputs a zero-velocity command instead of a valid control command, causing the robot to decelerate during normal operation and lowering its overall average navigation speed. (3) Incomplete Chain Instances, which

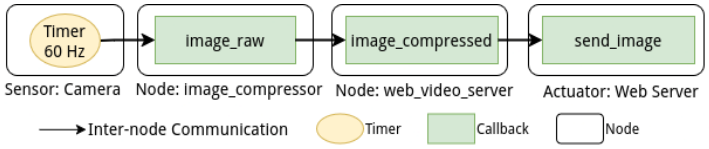


Fig. 13. Structure of the scenario-specific end-to-end chain cause-effect chain. The camera frame data is sampled and propagated to the web server, through `image_compressor` node located on Server 1 and `web_video_server` node on Server 2. This chain represents the pipeline of the camera video streaming.

indicate cases where sampled sensor data fails to propagate through the full chain. This typically results in wasted computation and missed updates to key components such as maps or control pipelines.

Additionally, we also monitor two functional performance metrics: (i) The task completion time required for the robot to reach its goal from the initial position, and (ii) The video streaming rate, ***frame per second*** (***FPS***) observed at Laptop 1, which serves as a quality indicator for end-to-end perceptual data transmission.

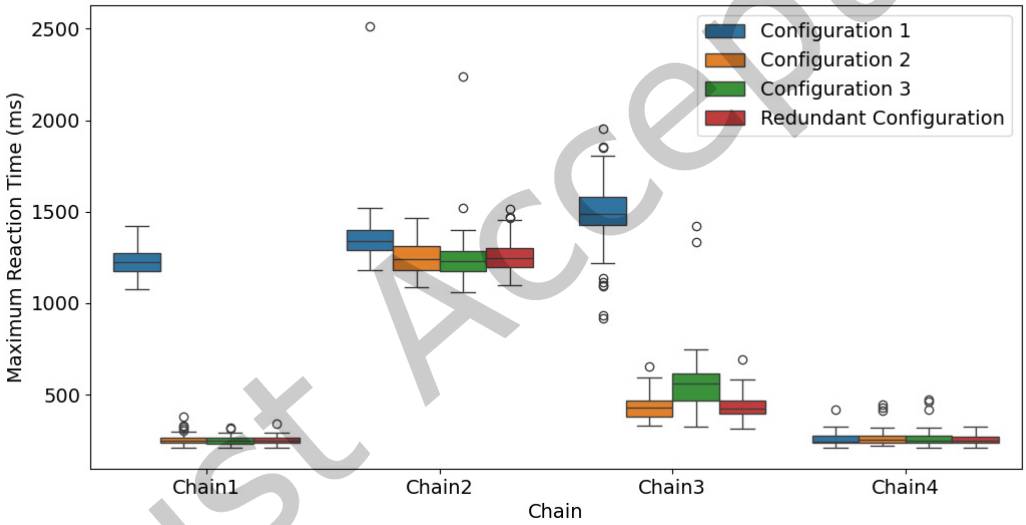


Fig. 14. Maximum reaction time for Chains 1-4 under different Configurations in Scenario 1. Chains 1 and 3 have significantly higher maximum reaction time under Configuration 1 compared to other configurations.

Experimental Results. Fig. 14 shows the Maximum Reaction Time for Chains 1–4 under different configurations. In Configuration 1, the co-location of `nav2_amcl`'s `scan` callback and `cv_detection` in the same ROS2 executor introduces significant delays due to ROS2 executor's non-preemptive scheduling. When `cv_detection` is running, `scan` must wait, resulting in a substantial increase in Chain 1's reaction time, as `follow_path` uses both old LiDAR data and new LiDAR data (explained in Section 4.3), with the latter being delayed. In contrast, Chain 2 is unaffected, since `compute_path_to_pose` only relies on older LiDAR data, roughly 1s earlier, which is already available in the TF buffer. This result confirms our observation in Section 4.3 that non-FIFO data access in real-world applications affects the real-time scheduling and analysis in ROS2.

Just Accepted

For Chain 3, although its scan callback of global_costmap is not on the same core of cv_detection, its calculation in compute_path_to_pose depends on transforming obstacle coordinates from the robot's base frame to the map frame, where the map frame is updated by nav2_amcl. Due to delayed execution of nav2_amcl, the most recent map frame often does not have an acceptable timestamp, so new obstacle coordinates cannot be transformed and may be discarded–resulting in increased reaction time and data loss. In comparison, the reaction time of Chain 4 remains stable under Configuration 1, as it operates in the odom frame, updated by the Odometry module rather than AMCL and unaffected by the contention.

Table 3. Deadline Miss and Sensor Data Loss per 100 chain instances of Chains 1-4 in Scenario 1

Chain	Configuration	Deadline Miss	Sensor Data Loss
Chain 1	Configuration 1	59 / 100	52 / 100
Chain 2	Configuration 1	0 / 100	52 / 100
Chain 3	Configuration 1	0 / 100	76 / 100
Chain 4	Configuration 1	0 / 100	0 / 100
Chains 1-4	Configuration 2-3, Redundant Configuration	0 / 100	0 / 100

Table 3 reports the number of deadline misses and sensor data loss events per 100 chain instances for Chains 1–4. In the Redundant Configuration, as well as Configurations 2 and 3, no deadline misses or data losses were observed. In contrast, Configuration 1 exhibits significant degradation due to the delayed execution of nav2_amcl's scan callback, which runs less frequently than the LiDAR sampling rate due to contention with cv_detection. As a result, many LiDAR samples are discarded before being processed, leading to high sensor data loss in Chains 1 and 2. As discussed previously, Chain 3 also suffers from data loss, as compute_path_to_pose discards updates from global_costmap when it cannot access a corresponding valid AMCL updates.

Table 4. Mission Completion Times for Configurations 1-3 and Redundant Configuration in Scenario 1

Configuration	Mission Completion Time (s)	
	maximum	average
Configuration 1	56.6	48.5
Configuration 2	40.3	39.4
Configuration 3	40.1	39.2
Redundant Configuration	41.3	39.7

Moreover, over half of Chain 1 instances miss the 1.2 s deadline, causing the robot to receive zero-velocity commands during the mission. This behavior increases the functional performance of average mission completion time from 39 s (in other configurations) to 48 s under Configuration 1, as shown in Table 4. These results reinforce our observation from Section 4.1 that missing deadlines in partial chains can directly impact functional-level performance and thus worth further research.

Finally, Fig. 15 shows the Reaction Time of the scenario-specific chain. In Configuration 3, Reaction Time starts around 140 ms and gradually drops to 40 ms by the 28-second of mission progress, while it remains around 40 ms under other configurations. This is because, in Configuration 3, the Global Planning thread–executing with higher priority and outside ROS2's control–preempts web_video_server, inflating Reaction Time of the scenario-specific chain. Additionally, the reduction in Reaction Time corresponds to the shortening execution time of Global Planning as the robot nears its goal–a behavior consistent with our observation in Section 4.2

Just Accepted

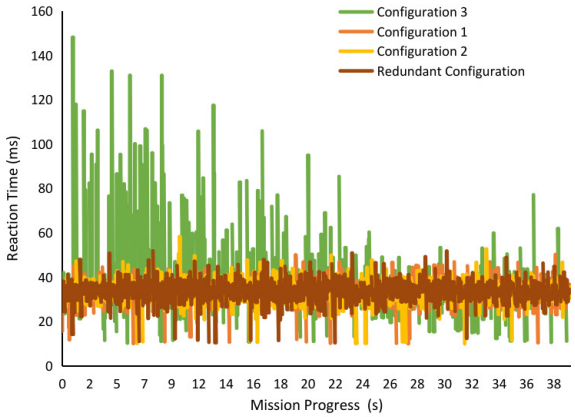


Fig. 15. Under diff configurations, reaction time of scenario-specific chain varies as mission progresses.

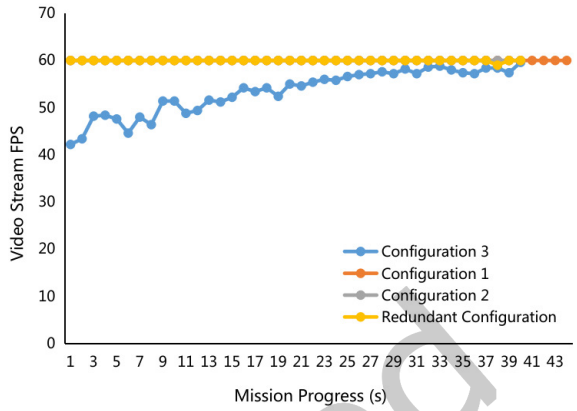


Fig. 16. Under diff configurations, video streaming rate (FPS) varies as mission progresses.

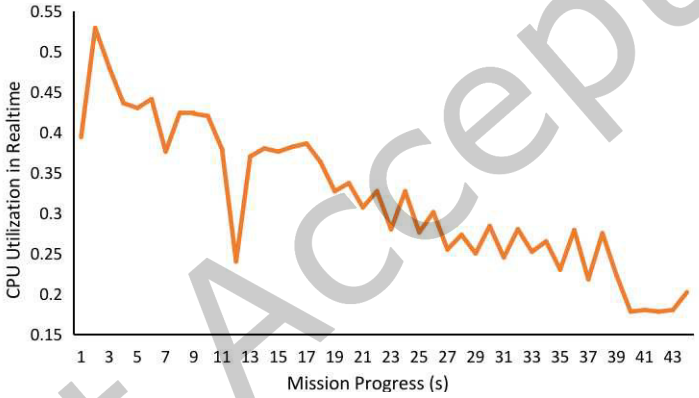


Fig. 17. CPU Utilization in Real-time on Core 5 under Configuration 3.

Just Accepted

that real-world callbacks may exhibit dynamically changing execution times. This trend is also reflected in the application's functional performance of the video streaming rate (FPS) shown in Fig. 16, where Configuration 3 shows a gradual improvement in streaming performance as the mission progresses. Note that the core utilization remains low even in the worst case for Configuration 3, as shown in the Fig. 17. These results highlight the runtime interference caused by threads uncontrolled by ROS2, as discussed in Section 4.4, and highlight the need for ROS2 timing analyses to account for such non-executor-controlled computations.

6.4 Application Scenario 2: Multi-Checkpoint Navigation with Perception and Logging

Application Scenario 2 presents a more complex task compared to Scenario 1. Instead of simple point-to-point navigation, the robot is required to traverse multiple checkpoints along a predefined route, from (0, 0), (6.43, 1.06), (6.21, -1.13), (6.29, -3.34), (5.96, -8.65), (3.81, -9.13), (1.51, -9.08), (-5.00, -7.37), back to (0, 0). At each checkpoint, the robot must save an image from its onboard camera to disk and simultaneously detect and count items on nearby shelves using onboard perception. This scenario more closely reflects the challenges of real-world robotic

applications, where multiple sensing, computation, and logging tasks must be performed concurrently and reliably.

Cause-Effect Chains under Consideration. In Scenario 2, Chains 1 to 4 remain the same as in Scenario 1. In addition, we consider a scenario-specific chain shown in Fig. 4, which starts from receiving a Goal Pose and ends when the corresponding Action Client receives a response from the Action Server. This partial chain reflects the responsiveness of Navigation2 in initiating goal-directed behavior and is subject to a hard real-time deadline of 20 ms, as discussed in Section 4.1.

Table 5. Response Time for Scenario-Specific Partial Cause-Effect Chain in Scenario 2

Configuration	**Response time (ms)**			
	maximum	average	98th percentile	99th percentile
Configuration 1	17.86518097	0.99155801	2.647032738	3.013119698
Configuration 2	67.94524193	1.10199214	2.988452911	13.45703125
Configuration 3	18.69606972	0.73312745	2.102632523	2.102632523
Redundant Configuration	15.02156258	0.80161039	2.635231018	3.146309853

Experimental Results. The Maximum Reaction Time, number of deadline misses, and amount of sensor data loss for Chains 1-4 remain the same as those in Scenario 1.

Table 5 presents the response times of the scenario-specific partial cause-effect chain. Configuration 2 exhibits a significantly higher maximum and 98th percentile response time compared to other configurations. This is due to contention on the executor when the robot reaches a checkpoint: the `camera_img_recorder` node writes an image to disk, while simultaneously the `goal_pose` callback receives the next navigation target. As explained in Section 4.2, receiving a navigation target changes the structure of the cause-effect chain from LiDAR to wheels from Fig. 7 to Fig. 6, activing global planning. As a result, the `write_img_to_disk` subscriber callback and the `handle_accept` action callback from `nav2_planner` are both activated and placed into the ReadySet of the same executor. Under the ROS2 executor's scheduling policy, subscriber callbacks have implicitly higher priority over action callbacks, causing `handle_accept` to be delayed and occasionally exceed the 20 ms deadline defined for this chain (as illustrated in Fig. 4). This leads to about 5 deadline misses in 100 instances under Configuration 2. However, this interference occurs only at checkpoints, and thus most response times under Configuration 2 remain comparable to those of other configurations, as reflected in the 98th percentile cutoff.

Furthermore, as discussed in Section 4.1, when the scenario-specific partial chain misses its 20 ms deadline, Navigation2's recovery mechanism is triggered. This can cause the system to enter a recovery behavior such as spinning or even aborting the mission. These results not only confirm the importance of real-time constraints on partial cause-effect chains in real-world ROS2 applications, but also highlight the observation in Section 4.2 that in real-world applications, the structure of cause-effect chains can change dynamically. Such structural changes can impact both the real-time behavior and functional performance of the system, reinforcing the need to account for these dynamics in the analysis and design of ROS2-based autonomous applications.

7 Case Study with Autoware Autonomous Driving System

To validate our observations in a more complex setting, we conduct a case study using the Autoware autonomous driving system. Compared to earlier robot experiments, Autoware features higher data throughput and more intricate cause-effect chains. In a lane-following task within Carla's Town01 simulation environment [11], we evaluate different configurations to observe how the identified timing and structural behaviors appear in real-world autonomous driving scenarios.

Just Accepted

7.1 Experimental Setup

The experimental setup uses two machines connected via a 1000 Mb/s Ethernet link. One runs Carla 0.9.15 with OpenCarla to provide ROS2-compatible simulation data; the other runs the Autoware.universe 2023.05 stack with ROS2 Humble. CUDA-dependent modules are disabled reduce variability, and `simple_trajectory_follower` is used for deterministic trajectory-following behavior suitable for controlled experiments. Experiments are conducted in Carla's Town01 map, where the vehicle performs a point-to-point lane-following task.

7.2 Scheduling Configurations

In the case study, we evaluate three scheduling configurations to assess their impact on the timing and functional behavior of the Autoware system. In the ***Best-Effort Configuration***, Autoware nodes are distributed across CPU cores in a balanced and predictable manner to avoid overloading. Four nodes with the highest CPU usage are each assigned to a dedicated core to prevent contention, with `pointcloud_container` placed on core 5, which is the core under analysis in all configurations. The remaining nodes are grouped by namespace and evenly distributed across cores 0–4, averaging about seven namespaces per core. All cores use the CFS for scheduling.

In ***Configuration 1***, we modify the Best-Effort setup by relocating the five nodes involved in Chains 1–3 under consideration (as shown in Fig. 18) to core 5, assigning each to a dedicated single-threaded executor. We apply real-time priorities as follows: `gyro_odometer` receives the highest priority, `stop_filter` a medium priority, and `ekf_localizer`, `ndt_scan_matcher`, and `simple_trajectory_follower` share the lowest priority, as detailed in Table 6. ***Configuration 2*** builds upon Configuration 1 by adding `imu_corrector` node (a sensor module involved in Chain 2) to core 5 and assigning it with high real-time priority, and also elevating the priorities of all nodes on core 5 except for `pointcloud_container` and `gyro_odometer`.

Table 6. Configuration 1

Node Name	Scheduler (Priority)
pointcloud_container	SCHED_FIFO (1)
ekf_localizer	SCHED_FIFO (1)
ndt_scan_matcher	SCHED_FIFO (1)
stop_filter	SCHED_FIFO (2)
gyro_odometer	SCHED_FIFO (3)
simple_trajectory_follower	SCHED_FIFO (1)

Table 7. Configuration 2

Node Name	Scheduler (Priority)
pointcloud_container	SCHED_FIFO (1)
ekf_localizer	SCHED_FIFO (2)
ndt_scan_matcher	SCHED_FIFO (2)
stop_filter	SCHED_FIFO (2)
gyro_odometer	SCHED_FIFO (2)
imu_corrector	SCHED_FIFO (2)
simple_trajectory_follower	SCHED_FIFO (3)

7.3 Cause-Effect Chains under Consideration.

In this case study, we focus on partial cause-effect chains, Chains 1–3 in Fig. 18, which differ in their upstream data sources (LiDAR, IMU, and Ego Status) but converge in `ekf_localizer` for pose and velocity fusion. The fused result is filtered by `stop_filter` and used by `simple_trajectory_follower` to compute control commands. These chains reflect the real-time performance of localization delivery essential for control. Alongside Maximum Reaction Time, we evaluate lane-keeping accuracy by examining trajectory deviations after turns as a direct visual indicator of control correctness.

7.4 Experimental Results

Table 8 presents the Maximum Reaction Time of Chains 1–3 under different configurations. In Configuration 1, the compute-intensive node `pointcloud_container` shares the same priority as key nodes on Chain 1, leading to resource contention. This causes a noticeable increase in Chain 1's Maximum Reaction Time, as pose messages

Just Accepted

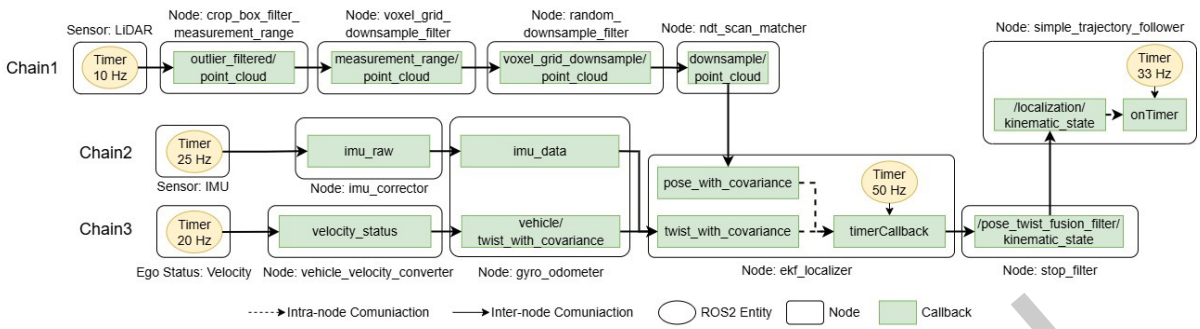


Fig. 18. Structure of Chains 1-3 in Autoware. Chain 1 starts from LiDAR through `ndt_scan_matcher` that processes filtered point cloud data and generates pose information for `ekf_localizer`; Chains 2 and 3 originate from IMU and Ego Status (velocity), respectively, where `gyro_odometer` computes vehicle velocity and provides twist information to `ekf_localizer`. The `ekf_localizer` node fuses pose and twist data via its `timerCallback` to produce final localization results, which are filtered by `stop_filter` and later accessed by `simple_trajectory_follower`. They capture the performance of localization feeding into control.

are delayed. In contrast, the upstream nodes of Chains 2 and 3 experience less interference due to being scheduled on other cores or having the highest priority on core 5. However, their downstream components still compete with `pointcloud_container`, resulting in moderate delays–less severe than in Chain 1.

Table 8. Reaction Time for Autoware Case Study

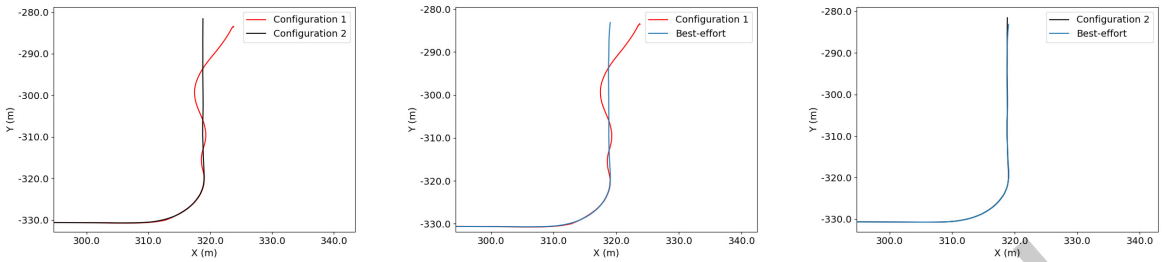
Chain	**Configuration**	**Maximum Reaction Time (ms)**			
		maximum	average	25th percentile	75th percentile
Chain 1	Configuration 1	1691.805537	347.134695	217.289092	443.655006
	Configuration 2	567.639075	144.744435	98.167509	177.671742
	Best-effort	370.431439	136.307050	108.591422	163.863284
Chain 2	Configuration 1	2330.677769	390.651151	246.201582	474.276933
	Configuration 2	1995.400829	231.918592	95.458657	287.906091
	Best-effort	3832.181475	341.092252	114.551161	417.925314
Chain 3	Configuration 1	1718.765890	304.888674	163.669578	372.198289
	Configuration 2	1483.957588	183.215799	93.294001	183.199094
	Best-effort	1873.675399	150.249289	96.616518	135.234724

Interestingly, the Maximum Reaction Time for Chains 2 and 3 in Configuration 1 is lower than in the Best-Effort Configuration. This is because, in the best-effort setup, nodes like `gyro_odometer` and `stop_filter` are scheduled using CFS on other cores, which introduces occasional additional delays in worst-case scenarios due to unpredictable contention.

In Configuration 2, nodes along Chains 1 and 3 all have higher priority than `pointcloud_container`, reducing the impact of interference. As a result, their Maximum Reaction Times are comparable to Best-Effort. For Chain 2, all involved nodes have higher priorities, allowing it to outperform the best-effort case where these nodes are CFS-scheduled and subject to potential interference.

This timing behavior ultimately impacts the functional performance of the system. The node `simple_trajectory_follower` relies on the Pose and Twist information produced by Chains 1–3 to compute control commands. In Configuration

Just Accepted



(a) Configurations 1 vs. 2 (b) Configuration 1 vs. Best-effort (c) Configuration 2 vs. Best-effort

Fig. 19. Vehicle's actual routes under pair-wise comparion of different configurations

1, the significantly higher Maximum Reaction Time of Chain 1 causes delayed delivery of the Pose input to `simple_trajectory_follower`, leading to degraded control accuracy. As a result, compared to Configuration 2 and the Best-Effort setup, the vehicle in Configuration 1 exhibits much larger lane deviation, producing an unstable trajectory that would be unacceptable in real-world applications.

Chains 1–3 are all partial cause-effect chains, each with different real-time requirements. In Autoware, as a representative autonomous system, the performance of these partial chains directly impacts the system's functional outcomes. By adjusting priority relationships based on Configuration 1, we observe significant improvements in both real-time metrics and functional performance, demonstrating the critical role of partial chain scheduling and analyses.

8 Conclusion

This work uncovers critical runtime behaviors in ROS2-based systems that affect both real-time guarantees and functional correctness in autonomous applications. Our empirical observations–covering partial chains, structural and timing variability, non-FIFO data access, and unmanaged threads–expose gaps in existing analysis models and scheduling strategies. We extend the ROS2 tracing toolchain to validate these findings through robotics and autonomous driving experiments. Future work will integrate these insights into formal timing models to enable more accurate real-time scheduling and analysis for complex ROS2 systems.

Acknowledgments

This work is supported by U.S. National Science Foundation (Grant No.CNS-2340171).

References

[1] Hazem Abaza, Debayan Roy, Shiqing Fan, Selma Saidi, and Antonios Motakis. 2024. Trace-enabled Timing Model Synthesis for ROS2-based Autonomous Applications. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.

[2] Abdullah Al Arafat, Sudharsan Vaidhun, Kurt M Wilson, Jinghao Sun, and Zhishan Guo. 2022. Response time analysis for dynamic priority scheduling in ROS2. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 301–306.

[3] Sinan Barut, Marco Boneberger, Pouya Mohammadi, and Jochen J Steil. 2021. Benchmarking real-time capabilities of ros 2 and orocos for robotics applications. In *International Conference on Robotics and Automation (ICRA)*. IEEE, 708–714.

[4] Christophe Bédard, Pierre-Yves Lajoie, Giovanni Beltrame, and Michel Dagenais. 2023. Message flow analysis with complex causal links for distributed ROS 2 systems. *Robotics and Autonomous Systems* 161 (2023), 104361.

[5] Christophe Bédard, Ingo Lütkebohle, and Michel Dagenais. 2022. ros2_tracing: Multipurpose low-overhead framework for real-time tracing of ROS 2. *Robotics and Automation Letters* 7, 3 (2022), 6511–6518.

Just Accepted

[6] Tobias Betz, Maximilian Schmeller, Harun Teper, and Johannes Betz. 2023. How fast is my software? latency evaluation for a ros 2 autonomous driving software. In *Intelligent Vehicles Symposium (IV)*. IEEE, 1–6.
[7] Tobias Blass, Arne Hamann, Ralph Lange, Dirk Ziegenbein, and Björn B Brandenburg. 2021. Automatic latency management for ROS 2: Benefits, challenges, and open problems. In *27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 264–277.
[8] Tobias Blaß, Daniel Casini, Sergey Bozhko, and Björn B. Brandenburg. 2021. A ROS 2 Response-Time Analysis Exploiting Starvation Freedom and Execution-Time Variance. In *2021 IEEE Real-Time Systems Symposium (RTSS)*. 41–53. doi:10.1109/RTSS52674.2021.00016
[9] Daniel Casini, Tobias Blaß, Ingo Lütkebohle, and Björn B. Brandenburg. 2019. Response-Time Analysis of ROS 2 Processing Chains Under Reservation-Based Scheduling. In *31st Euromicro Conference on Real-Time Systems (ECRTS 2019) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 133)*, Sophie Quinton (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 6:1–6:23. doi:10.4230/LIPIcs.ECRTS.2019.6
[10] Hyunjong Choi, Yecheng Xiang, and Hyoseung Kim. 2021. PiCAS: New design of priority-driven chain-aware scheduling for ROS2. In *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 251–263.
[11] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. 2017. CARLA: An Open Urban Driving Simulator. In *Proceedings of the 1st Annual Conference on Robot Learning*. 1–16.
[12] Tully Foote. 2013. tf: The transform library. In *2013 IEEE Conference on Technologies for Practical Robot Applications (TePRA)*. IEEE, 1–6.
[13] Mario Günzel, Harun Teper, Kuan-Hsun Chen, Georg von der Brüggen, and Jian-Jia Chen. 2023. On the equivalence of maximum reaction time and maximum data age for cause-effect chains. In *35th Euromicro Conference on Real-Time Systems (ECRTS 2023)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 10–1.
[14] Tobias Kronauer, Joshua Pohlmann, Maximilian Matthé, Till Smejkal, and Gerhard Fettweis. 2021. Latency analysis of ROS2 multi-node systems. In *international conference on multisensor fusion and integration for intelligent systems (MFI)*. IEEE, 1–7.
[15] Ruoxiang Li, Nan Guan, Xu Jiang, Zhishan Guo, Zheng Dong, and Mingsong Lv. 2022. Worst-case time disparity analysis of message synchronization in ROS. In *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 40–52.
[16] Ruoxiang Li, Xu Jiang, Zheng Dong, Jen-Ming Wu, Chun Jason Xue, and Nan Guan. 2023. Worst-Case Latency Analysis of Message Synchronization in ROS. In *Real-Time Systems Symposium (RTSS)*. IEEE, 185–197.
[17] Zihang Li, Atsushi Hasegawa, and Takuya Azumi. 2022. Autoware_Perf: A tracing and performance analysis framework for ROS 2 applications. *Journal of Systems Architecture* 123 (2022), 102341.
[18] Steve Macenski, Francisco Martín, Ruffin White, and Jonatan Ginés Clavero. 2020. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2718–2725.
[19] Hoora Sobhani, Hyunjong Choi, and Hyoseung Kim. 2023. Timing Analysis and Priority-driven Enhancements of ROS 2 Multi-threaded Executors. In *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. 106–118. doi:10.1109/RTAS58335.2023.00016
[20] Jinghao Sun, Tianyi Wang, Yang Li, Nan Guan, Zhishan Guo, and Guozhen Tan. 2023. SEAM: An Optimal Message Synchronizer in ROS with Well-Bounded Time Disparity. In *Real-Time Systems Symposium (RTSS)*. IEEE, 172–184.
[21] Yue Tang, Zhiwei Feng, Nan Guan, Xu Jiang, Mingsong Lv, Qingxu Deng, and Wang Yi. 2020. Response Time Analysis and Priority Assignment of Processing Chains on ROS2 Executors. In *2020 IEEE Real-Time Systems Symposium (RTSS)*. 231–243. doi:10.1109/RTSS49844.2020.00030
[22] Yue Tang, Nan Guan, Xu Jiang, Xiantong Luo, and Wang Yi. 2023. Real-time performance analysis of processing systems on ROS 2 executors. In *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 80–92.
[23] Yue Tang, Xu Jiang, Nan Guan, Xiantong Luo, Maolin Yang, and Wang Yi. 2024. Timing analysis of processing chains with data refreshing in ROS 2. *Journal of Systems Architecture* 155 (2024), 103259.
[24] Harun Teper, Mario Günzel, Niklas Ueter, Georg von der Brüggen, and Jian-Jia Chen. 2022. End-To-End Timing Analysis in ROS2. In *2022 IEEE Real-Time Systems Symposium (RTSS)*. 53–65. doi:10.1109/RTSS55097.2022.00015

Received 3 April 2025; revised 13 August 2025; accepted 4 October 2025

Just Accepted